



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Estudio e implementación paralela de métodos semiimplícitos multipaso

Resolución de Ecuaciones de Advección-Reacción-Difusión

Autor

José Manuel Navarro Cuartero

Director

José Miguel Mantas Ruiz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

**Estudio e implementación paralela de métodos
semiimplícitos multipaso:
Resolución de Ecuaciones de Advección-Reacción-Difusión**

José Manuel Navarro Cuartero

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

First name, Family name (student)

Keywords: Keyword1, Keyword2, Keyword3,

Abstract

Write here the abstract in English.

Yo, **José Manuel Navarro Cuartero**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75896777C, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Manuel Navarro Cuartero

Granada a X de Septiembre de 2026.

D. **José Miguel Mantas Ruiz**, Profesor del Área de XXXX del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado **Estudio e implementación paralela de métodos semiimplícitos multipaso, Resolución de Ecuaciones de Advección-Reacción-Difusión**, ha sido realizado bajo su supervisión por **José Manuel Navarro Cuartero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a X de Septiembre de 2026.

El director:

José Miguel Mantas Ruiz

Agradecimientos

Poner aquí agradecimientos...

Índice

Introducción	1
Conceptos previos	3
Métodos numéricos y EDOs	3
Tipos de métodos numéricos	4
Método de Euler	5
Paralelismo	7
Métodos de Resolución	10
Runge-Kutta	10
Adams-Bashford	12
Adams-Bashford-Moulton	14
Problemas	16
OpenMP	17
Implementación Secuencial	18
Paralelismo por Operaciones	19
Paralelismo por Componentes	21
Mediciones	22
CUDA	27
Implementación	30
Mediciones	35
Graphs	37
Shuffles y Grids multidimensionales	40
Bibliografía	45
Listado de Algoritmos	47
Listado de Figuras	48
Listado de Tablas	49

Introducción

Los métodos numéricos forman un conjunto de herramientas matemáticas cuyo propósito es aproximar soluciones allí donde las técnicas analíticas no alcanzan, ya sea debido a que la ecuación no admite solución cerrada, a que su forma es demasiado compleja o a que obtener la solución exacta sería inviable en términos de tiempo o recursos computacionales. En la práctica científica moderna, casi todos los modelos relevantes, físicos, biológicos, económicos, o de ingeniería, acaban desembocando en ecuaciones que describen cómo evoluciona un sistema, y esas ecuaciones rara vez pueden resolverse de forma analítica. Por eso, los métodos numéricos no son solo una alternativa; son el lenguaje cotidiano de la simulación y el cálculo aplicado.

Dentro de este ámbito de trabajo, las ecuaciones diferenciales ordinarias (EDOs) ocupan un lugar central. Una EDO es una relación entre una función desconocida y sus derivadas, y aparece siempre que un fenómeno depende de cómo cambia una cantidad respecto a otra: la velocidad de un objeto, la concentración de una sustancia, la temperatura en un punto, el crecimiento de una población. Resolver una EDO significa encontrar la función que satisface esa relación, pero en la mayoría de los casos esa función no puede escribirse de forma exacta. Incluso cuando existe una solución analítica, puede ser tan complicada que trabajar con ella resulte poco práctico. Por eso, recurrimos a métodos numéricos que construyen aproximaciones controladas y útiles.

El caso más habitual en aplicaciones es el problema de valor inicial (PVI). En un PVI no solo se especifica la ecuación diferencial, sino también el valor de la solución en un punto concreto. Esa condición inicial fija la trayectoria que debe seguir la solución y permite avanzar paso a paso, desde el punto conocido hacia los siguientes valores. Los métodos numéricos toman ese valor inicial y, mediante un proceso iterativo, generan una secuencia de aproximaciones en puntos igualmente espaciados. Cada paso intenta reproducir el comportamiento de la solución real dentro de un intervalo pequeño, y la calidad de esa aproximación depende del método elegido y del tamaño del paso.

Existen muchas formas de avanzar de un punto al siguiente, y cada una representa un compromiso entre precisión, estabilidad, coste computacional, y tiempo de ejecución. Es en este último punto en el que nos centraremos en los siguientes capítulos, ya que si bien para problemas pequeños, algo tan

simple como el método de Euler explícito podría bastarnos, posteriormente plantearemos problemas mucho más complejos, que requerirán soluciones que estén a la altura. Por tanto, no se trata solamente de hallar una solución, sino de hacerlo rápida y eficientemente.

Para esto, comenzaremos poniendo en contexto qué son los métodos numéricos y las ecuaciones diferenciales ordinarias, además de distinguir entre diversos tipos de métodos numéricos. También deberemos comprender la naturaleza de las dos tecnologías que implementaremos en nuestra búsqueda del paralelismo y la reducción de los tiempos de ejecución, a saber, OpenMP y CUDA, paralelismo a nivel de CPU y GPU.

También deberemos familiarizarnos extensamente con tres métodos de resolución en particular, y con los órdenes de los mismos. Usaremos Runge-Kutta, explícito y de un solo paso, Adams-Bashford, explícito y multipaso, y por último, Adams-Bashford-Moulton, semiimplícito y multipaso. Veremos sus ventajas y desventajas a la hora de elegir uno frente a otro.

Teniendo todo esto en cuenta, partiremos de 4 problemas cuya implementación secuencial y soluciones esperadas ya conocemos de forma previa, y aplicaremos para su resolución los tres métodos mencionados, que a su vez implementaremos con las dos tecnologías para paralelizar. Además, dentro de cada tecnología probaremos las distintas herramientas que ponen a nuestra disposición para enfocar la resolución de cada problema desde prismas diferentes.

Nuestro objetivo será implementar de forma fiel y exacta tanto los problemas como los métodos de resolución, para posteriormente poder comparar las ganancias en tiempo de ejecución al pasar de una tecnología a otra, o al aumentar el tamaño de los problemas, es decir, el número de ecuaciones con los que tratemos.

La idea detrás de todo esto es relativamente simple; un algoritmo que utilice 4 o 16 hebras será mejor que uno secuencial. Y uno que utilice 256 será mejor que todos ellos. Aunque la realidad no siempre va de la mano de nuestras ideas, haremos primero un estudio teórico teniendo en cuenta cómo funcionan las herramientas que utilizamos para después poder compararlo con las mediciones empíricas, y así poder validar, o desestimar, nuestras hipótesis.

Conceptos previos

Métodos numéricos y EDOs

Una ecuación diferencial es aquella que relaciona una función ($f(x)$ ó y), sus variables (x) y sus derivadas ($f'(x)$, dy/dx ó y'). Para resolver una ecuación diferencial se debe encontrar una función que satisfaga dicha ecuación. Dentro de las ecuaciones diferenciales nos encontramos las ecuaciones diferenciales ordinarias, EDOs, que contienen derivadas respecto a una sola variable independiente. Más adelante nos centraremos en este tipo en particular. En oposición a éstas estarían las ecuaciones diferenciales parciales, en las que la función $f(x)$ depende de más variables independientes. [Molero et al., 2007]

El orden de una ecuación diferencial equivale al orden de la mayor derivada que aparece en la ecuación. Además, será lineal cuando la variable dependiente y todas sus derivadas son de primer grado (no puede haber $f(x)^2$), y los coeficientes de la variable dependiente y sus derivadas dependen de la variable independiente.

A modo de ejemplo simple podríamos considerar una ecuación diferencial ordinaria de primer orden como:

$$\frac{dy}{dt} = -2y, y(0) = 1 \quad (1)$$

Donde además de la EDO, se nos está dando su valor inicial. En este caso en particular, sabemos de forma analítica que su solución exacta es $y(t) = e^{-2t}$, y podríamos por tanto calcular cualquier y_n que deseemos, pero si por cualquier razón no pudiéramos resolverla analíticamente deberíamos recurrir a los métodos numéricos. A través de ellos, y partiendo del valor y_n que se nos proporciona, calcularíamos de forma iterativa y_{n+1} hasta llegar al valor n que busquemos.

Podríamos utilizar, por ejemplo, el método de Euler, del que hablaremos más adelante. Antes de ello podemos entrar en algo más de detalle respecto a algunos otros conceptos que iremos usando.

Tipos de métodos numéricos

Podemos dividir a los métodos en tres grupos, explícitos, implícitos y semiimplícitos. La división se hará en función de cómo se consigue el siguiente paso, es decir, de cómo obtenemos y_{n+1} a partir de y_n ; de si aparecen valores futuros o no en la fórmula que lo calcula. Tendremos como expresión general la siguiente: [Molero et al., 2007]

$$y_{n+1} = y_n + \Delta t \cdot \Phi(x_n, y_n, y_{n+1}, \Delta t) \quad (2)$$

En la cual será la función incremento Φ la que dirima a cuál de los grupos pertenece el método con el que estemos trabajando.

Los métodos explícitos son aquellos en los que siguiente término se puede obtener directamente, sólo haciendo cálculos con datos de los que ya se dispone. Dicho de otra forma, son aquellos en los que y_{n+1} no aparece en Φ , sólo aparece en la parte izquierda de la ecuación. Entre ellos nos encontraremos el Euler explícito, Runge-Kutta, o Adams-Bashford, los tres los veremos posteriormente. Los métodos explícitos suelen ser más fáciles de implementar, ya que no requieren resolver ecuaciones, y suelen ser más rápidos. Por otro lado, tienden a ser menos estables, especialmente en problemas rígidos (*stiff*).

Al otro lado del espectro nos encontraríamos a los métodos implícitos, que son aquellos en los que y_{n+1} sí aparece en la parte de la derecha de la igualdad, lo cuál nos obligaría a resolver una ecuación que podrá ser, o no, lineal. Estos métodos son muy estables, y son más adecuados para resolver problemas rígidos (*stiff*). Como parte negativa tienen que requieren resolver ecuaciones en cada paso, y a veces de forma iterativa, lo que los acaba haciendo mucho más costosos computacionalmente. Entre ellos nos encontramos algunos métodos como el Euler implícito, Adams-Moulton, o los métodos BDF.

Como tercer grupo tendríamos los métodos semiimplícitos. Éstos representan una especie de "solución" a los diferentes problemas que traen consigo los métodos explícitos e implícitos. En este grupo, aunque existe una parte implícita, ésta es más simple, ya que o bien la ecuación a resolver es lineal, o bien se separa la parte rígida para tratarla implícitamente y el resto se trata de forma explícita. Esto es útil cuando el problema tiene términos rígidos (*stiff*) como la difusión, que deben tratarse implícitamente para que el método sea estable incluso con pasos de tiempo grandes, y términos suaves o de comportamiento hiperbólico como la advección, que se pueden tratar explícitamente para evitar resolver sistemas grandes o no lineales. Los métodos semiimplícitos representan un compromiso entre la estabilidad de los explícitos y la velocidad de los implícitos. Como hemos mencionado, podremos dividir a la EDO en dos partes:

$$y' = f(x, y) = f_E(x, y) + f_I(x, y) \quad (3)$$

En este ejemplo f_E es la parte explícita, suave (no *stiff*), y f_I es la parte que queremos tratar implícitamente. Algunos métodos semiimplícitos son los métodos IMEX (*implicit-explicit*) [Ascher et al., 1995], los métodos linealmente implícitos Runge-Kutta (*LIRK*) [Calvo et al., 2001], o el método Adams-Bashford-Moulton (AB-AM predictor-corrector). Es con este último con el que trabajaremos.

Otra categorización con la que trabajaremos serán los métodos de un solo paso, y los métodos multipaso. En los métodos de un solo paso, para obtener el siguiente valor con el que se tratará, el siguiente paso, sólo se utiliza información relativa al paso anterior. Es decir, como podemos ver en [2](#), para obtener y_{n+1} sólo se utilizaría y_n .

Sin embargo, los métodos multipaso usan varios valores anteriores de la solución para calcular el siguiente paso en el tiempo, en lugar de basarse solo en el valor actual. Es decir, que para obtener y_{n+1} se podrían tomar pasos intermedios que posteriormente se desechen (como en Runge-Kutta), o utilizar pasos más antiguos como y_{n-1} o y_{n-2} . Para problemas de gran tamaño, los métodos multipaso permiten alcanzar resultados con gran precisión y de una forma más eficiente que los métodos de un paso como el de Euler o el de Runge-Kutta.

Método de Euler

Como hemos comentado, el método de Euler es uno de los métodos numéricos más simples de los que disponemos para la resolución de EDOs. Se trata de un método explícito de un solo paso que se construye a partir de una sencilla idea geométrica: aproximar el valor de la solución en cada nodo por el valor que proporciona la recta tangente a la solución trazada desde el punto correspondiente al nodo anterior. [Gallardo, 2018]

Emplearíamos la siguiente fórmula:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) \quad (4)$$

Aplicando la misma al ejemplo de valor inicial que mencionamos antes, $\frac{dy}{dt} = -2y$, $y(0) = 1$, sabemos que y_n corresponde a la aproximación numérica de $y(t_n)$, y Δt representa la variación en el tiempo paso a paso que estemos considerando. Además, en nuestro caso, $f(t_n, y_n)$ sería $-2 \cdot y_n$, y se nos da el valor inicial $t_0 = 0.0$. Por tanto, tomando estos valores, con un $\Delta t = 0.1$, y $t_f = 1.0$, calcularemos los sucesivos valores para $t = 0.1, 0.2, \dots, 1.0$.

El primer paso sería por tanto, partiendo de los valores iniciales, calcular el $t = 0.1$, a saber, $y_1 = y_0 + \Delta t \cdot f(t_0, y_0) = 1 + 0.1 \cdot (-2 \cdot 1) = 0.8$. Si esto lo repitiéramos de forma sucesiva obtendríamos los siguientes resultados, los

cuales además compararemos con el valor real $y(t) = e^{-2t}$ para obtener el error absoluto ε_a y el error relativo ε_r . Definiremos al error absoluto como la diferencia entre el valor real y nuestra aproximación, y al error relativo como el cociente entre el error absoluto y el valor real, en porcentaje.

Paso n	t_n	$y_n(\text{Euler})$	$y_n(\text{Exacta})$	ε_a	ε_r
0	0.0	Valor Inicial = 1.0		-	-
1	0.1	0.8	0.81873	0.01873	2.29
2	0.2	0.64	0.67032	0.03032	4.52
3	0.3	0.512	0.54881	0.03681	6.71
4	0.4	0.4096	0.44933	0.03973	8.84
5	0.5	0.32768	0.36788	0.04020	10.93
6	0.6	0.26214	0.30119	0.03905	12.97
7	0.7	0.20972	0.24660	0.03688	14.96
8	0.8	0.16777	0.20190	0.03412	16.90
9	0.9	0.13422	0.16530	0.03108	18.80
10	1.0	0.10737	0.13534	0.02796	20.66

Tabla 1: Comparativa entre Euler y valor real

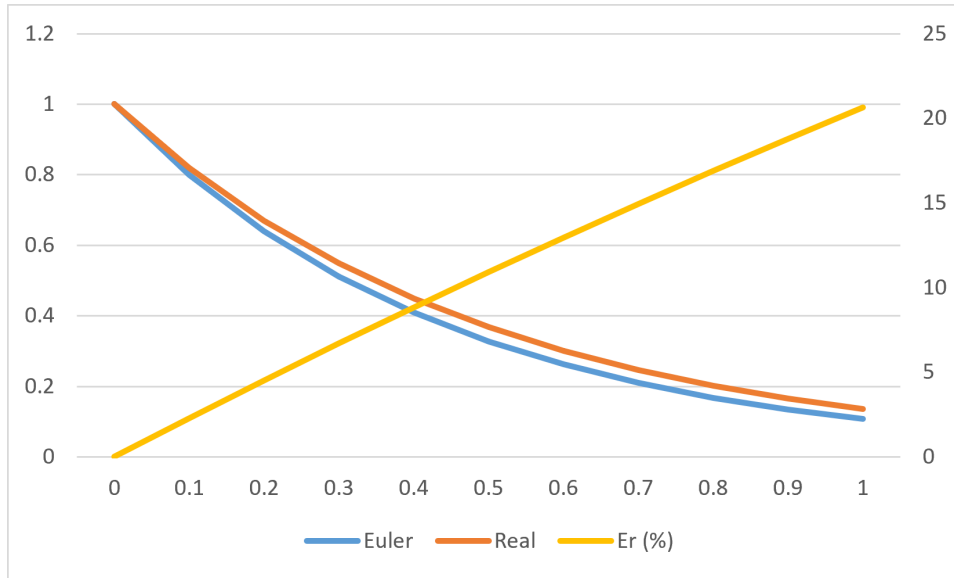


Figura 1: Comparativa entre Euler y valor real

Como podemos observar en la tabla 1 y en la figura 1, los valores que obtenemos con el Euler explícito son próximos a los reales (eje Y izquierdo); el error absoluto se mantiene estable por debajo de 0.05, e incluso disminuye. Sin embargo, el error relativo (eje Y derecho) aumenta de forma lineal con

cada iteración, y para la última alcanza un preocupante 20 %. Esto nos indica que para este problema, quizás el método numérico que hemos utilizado o el Δt que hemos considerado no son los más indicados.

Paralelismo

Las dos tecnologías que vamos a utilizar para introducir el paralelismo en nuestro código y las mejoras que trae consigo son OpenMP y CUDA, es decir, paralelismo a nivel de CPU, y a nivel de GPU, respectivamente.

Por muy rápida que sea nuestra CPU, siempre llegaremos a un punto en que, bien por la complejidad de los problemas, bien por el tamaño que se maneje, una sola CPU con un código secuencial no va a ser suficiente. Es aquí donde entra el paralelismo, y es aquí donde nos centraremos.

La programación paralela es el conjunto de técnicas, modelos y arquitecturas que permiten ejecutar múltiples operaciones de forma simultánea, explotando el hardware moderno (CPU multicore, GPU masivamente paralelas, aceleradores, clústeres). Su objetivo es reducir el tiempo de ejecución, aumentar el rendimiento y permitir resolver problemas que, como hemos comentado, serían inviables en código secuencial.

El paralelismo surge cuando un problema puede descomponerse en sub-tareas independientes que pueden ejecutarse al mismo tiempo. En términos formales, un programa es paralelizable si existe una partición de su espacio de trabajo en unidades que no presentan dependencias de datos que impidan su ejecución concurrente. Esto implica analizar dependencias de datos, localidad y acceso a memoria, granularidad de las tareas, y coste de sincronización.

Si quisiéramos clasificar los distintos modelos de paralelismo nos podríamos encontrar, por un lado con el paralelismo de datos, que consiste en aplicar la misma operación a múltiples elementos de un conjunto, a través de la vectorización, de bucles paralelos, o de operaciones *element-wise*. Dentro de esta categoría nos encontraríamos a OpenMP, CUDA, OpenCL, o AVX, entre otros.

Por otro lado tendríamos el paralelismo de tareas, que consiste en ejecutar tareas heterogéneas en paralelo, como pipelines, árboles de tareas, o grafos de dependencias. Este modelo es usado por frameworks como TBB, Cilk, OpenMP tasks o runtimes de grafos. Por último el paralelismo híbrido combina ambos, buscando el paralelismo de tareas, con paralelismo de datos dentro de cada tarea.

Algunos conceptos muy importantes que nos encontraremos son la concurrencia y el paralelismo. La concurrencia hace referencia a que varias tareas pueden ocurrir a la vez, o que ocurren simultáneamente de forma lógica. Una CPU *single-core* puede cambiar rápidamente de una tarea a otra si no tienen dependencia entre ellas y son concurrentes. Mientras que el paralelismo

trata sobre ejecutar físicamente varias tareas al mismo tiempo, por ejemplo a través de varios núcleos (como en OpenMP) o de varias hebras (como en CUDA).

El speedup será la forma en la que mediremos las mejoras en la ejecución conforme aumenta el paralelismo. En nuestro caso, lo calcularemos dividiendo el tiempo de ejecución del programa secuencial, entre el paralelizado. [Schryen, 2024] Como todo código por bien optimizado que esté va a tener una parte secuencial que no pueda paralelizarse, buscaremos que el speedup aumente tanto como sea posible. Por ejemplo, pasando de un código secuencial a uno con 4 hebras OpenMP, el speedup ideal sería de 4, aunque entendemos que no será posible conseguirlo, pero sí acercarse lo suficiente.

Otro concepto relacionado sería el de la eficiencia, que sería el cociente entre speedup y el número de cores, o de hebras, empleado para obtener ese speedup. La eficiencia estará entre 0 y 1 (o entre 0 % y 100 %), y medirá cuánto se están aprovechando los recursos paralelos.

Al hilo de esto último, la Ley de Amdahl [Amdahl, 1967] nos indica que, con un tamaño de problema constante y aumentando el número de procesadores, la parte no paralelizable del programa va a limitar nuestro speedup. Podemos ilustrarla tal que:

$$S(p) = \frac{1}{(1 - f) + f/p} \quad (5)$$

Siendo p el numero de procesadores, f la fracción paralelizable del programa, $1 - f$ por tanto sería la parte secuencial, y $S(p)$ sería el speedup máximo teórico. La interpretación que podemos hacer es que cuando aumentan los hilos, la parte no paralelizable satura el speedup. Por ejemplo, con un 95 % del programa paralelizable, e infinitos procesadores, el speedup nunca superará 20. Trayendo esto a un contexto más realista, lo que se nos indica es que, aunque obviamente aumentar los núcleos o los hilos es efectivo, y reduciremos los tiempos de ejecución, llegará un número a partir del cual ya no merezca la pena.

También debemos mencionar la Ley de Gustafson [Gustafson, 1988], que parte de una premisa distinta, aumentar el tamaño del problema cuando tenemos más procesadores. A saber:

$$S(p) = p - (1 - f) \cdot (p - 1) \quad (6)$$

Aquí la interpretación que hacemos es que al aumentar el tamaño del problema, aumentamos la parte paralelizable, mientras que la secuencial se mantiene estable. Esto juega a nuestro favor, ya que trataremos grandes cantidades de datos. La ley de Gustafson aporta un punto de vista más optimista al binomio procesadores-eficiencia, ya que nos dice que aunque la eficiencia no pueda ser perfecta, a medida que el problema y los hilos crezcan, podrá acercarse mucho.

Otro concepto importante será el de la sincronización. Cuando hilos diferentes deban compartir algún dato entre ellos, o necesiten que haya terminado cierta tarea antes de empezar la siguiente, deberá haber una barrera en la que los hilos se esperen entre ellos. Estas barreras ralentizan mucho la ejecución y se deberá intentar reducirlas tanto como sea posible.

Métodos de Resolución

Para resolver los diferentes problemas hemos implementado tres métodos distintos, a saber: Runge-Kutta, que es explícito y utiliza pasos intermedios ($K1, K2$) para obtener los nuevos pasos; Adams-Bashford, que también es explícito y multipaso, pero utiliza pasos antiguos (como f_{n-1} o f_{n-2}); y Adams-Bashford-Moulton, un predictor-corrector semiimplícito multipaso basado en Adams-Moulton (formalmente implícito) que utiliza Adams-Bashford para predecir y_{n+1} y sustituirlo en $f(t_{n+1}, y_{n+1})$.

Runge-Kutta

Ahora que hemos tratado el método de Euler, podremos dar el siguiente paso hacia Runge-Kutta. Aunque técnicamente ya lo hemos estado viendo, ya que los métodos de Runge-Kutta son generalizaciones de la fórmula básica de Euler. [Segarra-Escandon, 2020] Por esta razón, se dice que el método de Euler es un método de Runge-Kutta de primer orden.

En cada uno de los órdenes de Runge-Kutta resolveremos una serie de pasos intermedios para hallar el valor final. El de orden 1, como hemos mencionado, coincidiría con Euler, como se puede ver en la ecuación 4.

Para ilustrar este punto, procedemos a pasar a la fórmula de segundo orden, en el que buscaremos encontrar constantes o parámetros w_1, w_2, α, β tal que la fórmula concuerda con un polinomio de Taylor de grado 2. [Zill, 2009]

$$\begin{aligned}y_{n+1} &= y_n + h \cdot (w_1 k_1 + w_2 k_2) \\k_1 &= f(x_n, y_n) \\k_2 &= f(x_n + \alpha h, y_n + \beta h k_1)\end{aligned}\tag{7}$$

Para el uso que haremos en este proyecto, basta decir que esto se puede hacer siempre que las constantes cumplan que:

$$\begin{aligned}w_1 + w_2 &= 1, \quad w_2 \alpha = \frac{1}{2} \text{ y } w_2 \beta = \frac{1}{2} \\w_1 &= 1 - w_2, \quad \alpha = \frac{2}{2w_2} \text{ y } \beta = \frac{1}{2w_2}\end{aligned}$$

Este es un sistema algebraico que posee tres ecuaciones y cuatro incógnitas, lo cual implica un número infinito de soluciones, donde $w_2 \neq 0$, y para nuestra aplicación práctica tomaremos $w_1 = w_2 = \frac{1}{2}$ y $\alpha = \beta = 1$, formando por tanto:

$$\begin{aligned} y_{n+1} &= y_n + h/2 \cdot (k_1 + k_2) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h, y_n + k_1 \cdot h) \end{aligned} \tag{8}$$

De forma análoga, obtendremos el tercer orden:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 4k_2 + k_3) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h, y_n - k_1 \cdot h + 2k_2 \cdot h) \end{aligned} \tag{9}$$

En el método de Runge-Kutta de orden 4 resolveremos diversos pasos que nos servirán para estimar de forma muy aproximada el resultado. Será en éste en el que nos centraremos a la hora de las mediciones. Partiendo de un valor inicial $dy/dt = f(t, y)$, $y(t_0) = y_0$, y de un valor de salto $h > 0$, calcularemos iterativamente los valores tal que:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h/2, y_n + k_2 \cdot h/2) \\ k_4 &= f(t_n + h, y_n + k_3 \cdot h) \end{aligned} \tag{10}$$

Aplicado en un contexto de programación, tendremos una clase *RungeKutta* con una función principal que aplicará el cálculo y algunas auxiliares que se explicarán más adelante. Dicha función principal tendrá el siguiente diseño, en el que se harán diversas llamadas a la evaluación de la derivada de cada problema en cuestión (aquí lo llamaremos *feval*):

Algoritmo 1 Runge-Kutta de Orden 4

```
 $Y_n \leftarrow Y_0$   
 $t_n \leftarrow t_0$   
while  $t_n < t_f$  do  
   $K1 \leftarrow f_{eval}(t_n, Y_n)$   
   $K2 \leftarrow f_{eval}(t_n + h/2, Y_n + K1 \cdot h/2)$   
   $K3 \leftarrow f_{eval}(t_n + h/2, Y_n + K2 \cdot h/2)$   
   $K4 \leftarrow f_{eval}(t_n + h, Y_n + K3 \cdot h)$   
   $Y_n \leftarrow Y_n + h/6 \cdot (K1 + 2 \cdot K2 + 2 \cdot K3 + K4)$   
   $t_n \leftarrow t_n + h$   
end while  
 $Y_f \leftarrow Y_n$ 
```

Como podemos ver, la aplicación del algoritmo es relativamente fácil de entender, se trabaja con 4 vectores auxiliares en los que hacemos los cálculos intermedios, que luego se suman ponderadamente. Como cada evaluación K depende de la anterior, no podremos hacer una operación vectorial en la que se evalúen los cuatro K en paralelo, sino que será en las otras operaciones, las evaluaciones de cada miembro, las sumas, y las multiplicaciones de cada vector, en las que podremos entrar a trabajar el paralelismo para mejorar la eficiencia de nuestro código.

Adams-Bashford

A diferencia de Runge-Kutta, donde sólo considerábamos un valor y_n para obtener y_{n+1} , en los métodos de Adams-Bashford, al ser multipaso, sí tendremos en cuenta más puntos, como y_{n-1} o y_{n-3} . En este proyecto nos hemos centrado en los métodos de paso fijo, es decir, que la diferencia entre y_n y y_{n-1} es constante (e igual a Δt). Si bien existen otros métodos de paso variable, no serán tratados.

De forma general, tenemos que un método lineal de k pasos viene determinado por una expresión tal que: [Molero et al., 2007]

$$\sum_{j=0}^k \alpha_j \cdot z_{n+j} = h \cdot \sum_{j=0}^k \beta_j \cdot f(x_{n+j}, z_{n+j}) , n \geq 0 \quad (11)$$

Para poder aplicar esta fórmula necesitaremos unos valores de arranque, que serán iguales al valor de k . Como en los problemas de valor inicial sólo se proporciona el primero de ellos y_0 , los valores restantes deberemos conseguirlos aplicando otro método (que no sea multipaso), como por ejemplo, Runge-Kutta.

La expresión general de un método de Adams-Bashford de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_{k-1} \cdot \nabla^{k-1}) f_n$$

donde

$$\gamma_i = \frac{1}{i! h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_n) \dots (x - x_{n-i+1}) \cdot dx \quad (12)$$

Aplicando la anterior ecuación 12, obtendríamos los distintos órdenes con los que trabajaremos. Al igual que con Runge-Kutta, el Adams-Bashford de orden 1 coincide con Euler explícito (ecuación 4). De igual manera, obtendríamos Adams-Bashford de 2 pasos:

$$y_{n+1} = y_n + h/2 \cdot (3f_n - f_{n-1}) \quad (13)$$

Este será el primer caso en el que vemos aplicada la teoría de los métodos multipaso, ya que, para obtener el paso y_2 , por ejemplo, deberíamos tener de antemano f_1 y f_0 , para los que a su vez necesitaríamos (a través del arranque con Runge-Kutta) y_1 y y_0 . Una vez hecho el arranque inicial para obtener y_2 , ya sí podríamos operar iterativamente sólo con Adams-Bashford. El de orden 3 sería:

$$y_{n+1} = y_n + h/12 \cdot (23f_n - 16f_{n-1} + 5f_{n-2}) \quad (14)$$

Y por último tendríamos la expresión de orden 4, que será con la que principalmente trabajaremos:

$$y_{n+1} = y_n + h/24 \cdot (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \quad (15)$$

Llevándonos este método a la programación, tendremos una clase *Adams-Bashford* con una función *aplicar* que se encargará de, primero, realizar un arranque con Runge-Kutta de orden 4 (algoritmo 1) y posteriormente poner en funcionamiento un bucle que ejecute la expresión tantas veces como esté establecido. En función del lenguaje utilizado y del tipo de paralelismo, las operaciones vectoriales funcionarán de una forma u otra, pero el planteamiento será el mismo, con llamadas a la evaluación de cada problema en cuestión (*feval*). El algoritmo para aplicar el método, siendo k el orden del mismo, será:

Algoritmo 2 Adams-Bashford general

```
 $Y_n \leftarrow \text{arranqueRK4}(y_{n-1}) \quad \triangleright 1 \leq n < k$   
 $F_n \leftarrow \text{feval}(Y_n) \quad \triangleright 0 \leq n < k$   
 $t_n \leftarrow t_0 + (k-1) \cdot h$   
while  $t_n < t_f$  do  
     $Y_k \leftarrow \text{AdamsBashford-K}(Y_{k-1}, F_n) \quad \triangleright 0 \leq n < k$   
     $F_k \leftarrow \text{feval}(Y_k)$   
     $\text{swap}(Y_k, Y_{k-1})$   
     $\text{swap}(F_n, F_{n-1}) \quad \triangleright 1 \leq n \leq k$   
     $t_n \leftarrow t_n + h$   
end while  
 $Y_f \leftarrow Y_{k-1}$ 
```

La idea detrás del algoritmo es que, tras el arranque de los $k-1$ vectores y_n , y la evaluación de los k vectores f_n , donde k es igual al orden de A-B que estemos aplicando, entrar en el bucle iterativo. En él, iteraremos sobre 2 vectores y_n , a saber, y_k y y_{k-1} , y sobre $k+1$ vectores f_n .

Una vez en el bucle, calculamos el nuevo valor de y_k , y con él, f_k . Posteriormente se hará un swap de vectores tal que y_k pasa a ser y_{k-1} , y todos los vectores f_n pasan a ser f_{n-1} . Así tendremos los datos preparados para la siguiente iteración.

Adams-Bashford-Moulton

Los métodos de Adams-Moulton tienen un funcionamiento similar a los de Adams-Bashford, con la salvedad de que estos son implícitos. Esto quiere decir que para calcular y_{n+1} se hace uso de $f(t_{n+1}, y_{n+1})$. Para resolver este problema utilizaremos un sistema Predictor-Corrector dándonos un método semiimplícito.

La expresión general de un método Adams-Moulton puro de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_k \cdot \nabla^k) f_{n+1}$$

donde

$$\gamma_0 = 0 \text{ y } \gamma_i = \frac{1}{i!h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_{n+1}) \dots (x - x_{n-i+2}) \cdot dx \quad (16)$$

Y por tanto, las ecuaciones correspondientes a los mismos son las siguientes: [Segarra-Escandon, 2020]

Adams-Moulton orden 0 (AM1)

$$y_n = y_{n-1} + h \cdot f_n \quad (17)$$

Adams-Moulton orden 1 (AM2)

$$y_{n+1} = y_n + h/2 \cdot (f_{n+1} + f_n) \quad (18)$$

Adams-Moulton orden 2 (AM3)

$$y_{n+1} = y_n + h/12 \cdot (5f_{n+1} + 8f_n - f_{n-1}) \quad (19)$$

Adams-Moulton orden 3 (AM4)

$$y_{n+1} = y_n + h/24 \cdot (9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \quad (20)$$

Adams-Moulton orden 4 (AM5)

$$y_{n+1} = y_n + h/720 \cdot (251f_{n+1} + 646f_n - 264f_{n-1} + 106f_{n-2} - 19f_{n-3}) \quad (21)$$

Será en el AM5 en el que nos centraremos principalmente.

Es intuitivamente obvio que llegados a este punto no podríamos resolver estos métodos con las herramientas de las que disponemos. ¿Cómo llegamos a f_{n+1} sin disponer todavía de y_{n+1} Para resolver este problema, empleamos un sistema Predictor-Corrector mediante el cual utilizamos Adams-Bashford para hacer una primera aproximación de f_{n+1} , y la evaluamos para posteriormente hacer una segunda aproximación usando Adams-Moulton. El esquema será el siguiente, donde k es igual al orden:

Algoritmo 3 Adams-Bashford-Moulton general

$Y_n \leftarrow \text{arranqueRK4}(Y_{n-1})$	$\triangleright 1 \leq n < k$
$F_n \leftarrow \text{feval}(Y_n)$	$\triangleright 0 \leq n < k$
$t_n \leftarrow t_0 + (k-1) \cdot h$	
while $t_n < t_f$ do	
$Y_{pred} \leftarrow \text{AdamsBashford-K}(Y_{k-1}, F_{k-1})$	$\triangleright$ Aplicar orden k
$F_{pred} \leftarrow \text{feval}(t_{n+h}, Y_{pred})$	
$Y_{corr} \leftarrow \text{AdamsMoulton-K}(Y_{k-1}, F_{pred})$	$\triangleright$ Aplicar orden k
$F_{corr} \leftarrow \text{feval}(t_{n+h}, Y_{corr})$	
$Y_k \leftarrow \text{AdamsMoulton-K}(Y_{k-1}, F_{corr})$	$\triangleright$ Aplicar orden k
$F_k \leftarrow \text{feval}(t_{n+h}, Y_k)$	
$\text{swap}(y_k, y_{k-1})$	
$\text{swap}(f_n, f_{n-1})$	$\triangleright 1 \leq n \leq k$
$t_n \leftarrow t_n + h$	
end while	
$Y_f \leftarrow Y_{k-1}$	

El funcionamiento del algoritmo acaba siendo muy similar al esquema de Adams-Bashford, en tanto que hacemos el arranque con Runge-Kutta y posteriormente iteramos sobre los vectores Y_n y F_n . En este caso, añadimos además los vectores predictor y corrector, Y_{pred} y F_{corr} , que nos permiten aplicar la idea central de este método, manteniendo una buena eficiencia pero con resultados más precisos.

Problemas

Para poner en funcionamiento la implementación de estos métodos y de la paralelización de la que hablaremos a continuación, hemos tomado cuatro problemas distintos que nos permitan probar extensivamente ambas tecnologías, tanto OpenMP como CUDA. Los problemas en cuestión son:

SimpleAdvDiff y AdvDiff1D son problemas unidimensionales de advección - difusión en los que se trabajará con vectores que tendrán un número de EDOs igual al tamaño del vector y estarán formados por dos partes, una rígida (*stiff*) y una no rígida. Si bien servirán para establecer las bases del proyecto, nos centraremos más en los otros dos problemas.

Brusselator1D, el modelo brusselator es un tipo de modelo de reacción-difusión que fue desarrollado por la Brussels School. Este sistema está diseñado para analizar el comportamiento de sistemas químicos que presentan oscilación no lineal. [Mara'beh et al., 2024] Este problema tendrá 2 componentes, y por tanto el número de EDOs será el doble del tamaño que se elija.

Brusselator2D, este modelo es similar a su versión unidimensional, pero incorpora una componente extra reactiva. [Mara'beh et al., 2024] Además, está discretizado uniformemente en una malla de $N \times N$ puntos, por lo tanto, su número de EDOs será el doble del cuadrado del tamaño de problema.

OpenMP

OpenMP es un modelo de programación paralela para arquitecturas de memoria compartida, diseñado para proporcionar una interfaz portable, escalable y de alto nivel que permita explotar el paralelismo en CPUs multinúcleo sin recurrir a APIs de hilos de bajo nivel. Su diseño se basa en directivas de compilador, rutinas de biblioteca y variables de entorno, y sigue el modelo de ejecución *fork-join*, donde un hilo maestro crea y sincroniza equipos de hilos que ejecutan regiones paralelas. [Jin et al., 2024]

Surge en 1997, cuando Intel impulsa la creación del OpenMP Architecture Review Board (ARB), un consorcio formado por fabricantes de hardware y software (Intel, IBM, AMD, HP, Nvidia, Oracle, entre otros) con el objetivo de definir un estándar común para paralelismo en memoria compartida. La última versión de OpenMP, la 6.0, data de 2024.

OpenMP implementa paralelismo mediante *multithreading*. El funcionamiento es relativamente simple, el programa comienza con un comportamiento secuencial y un hilo maestro. Cuando este hilo se encuentra una región paralela con `#pragma omp parallel`, se crea un equipo de hilos [ARB, 2024]. Los hilos ejecutan el bloque paralelo de forma concurrente, y al finalizar, los hilos se unen y el control vuelve al maestro.

Podemos identificar se compone de varios elementos principales, el primero de los cuales sería las directivas de compilador, como `#pragma omp parallel`, `#pragma omp for`, o `#pragma omp single`, y sus cláusulas como `schedule`, `shared`, o `default`. Otros elementos importantes son las funciones de biblioteca, como `omp_set_num_threads()` o `omp_get_wtime()`, y las variables de entorno, como `OMP_NUM_THREADS`, `OMP_SCHEDULE`, o `OMP_DYNAMIC`. [ARB, 2024]

Como hemos comentado, OpenMP opera sobre un modelo de memoria compartida en la que todos los hilos ven el mismo espacio de direcciones, en el que las variables pueden ser visibles por todos los hilos (*shared*), o cada hilo tiene su propia copia (*private*). Este modelo evita la necesidad de comunicación explícita entre procesos (como en MPI), simplificando el desarrollo. [Jin et al., 2024]

Implementación Secuencial

Tomaremos como punto de partida la implementación secuencial de los problemas [Mantas, 2024], de la cual ya disponemos.

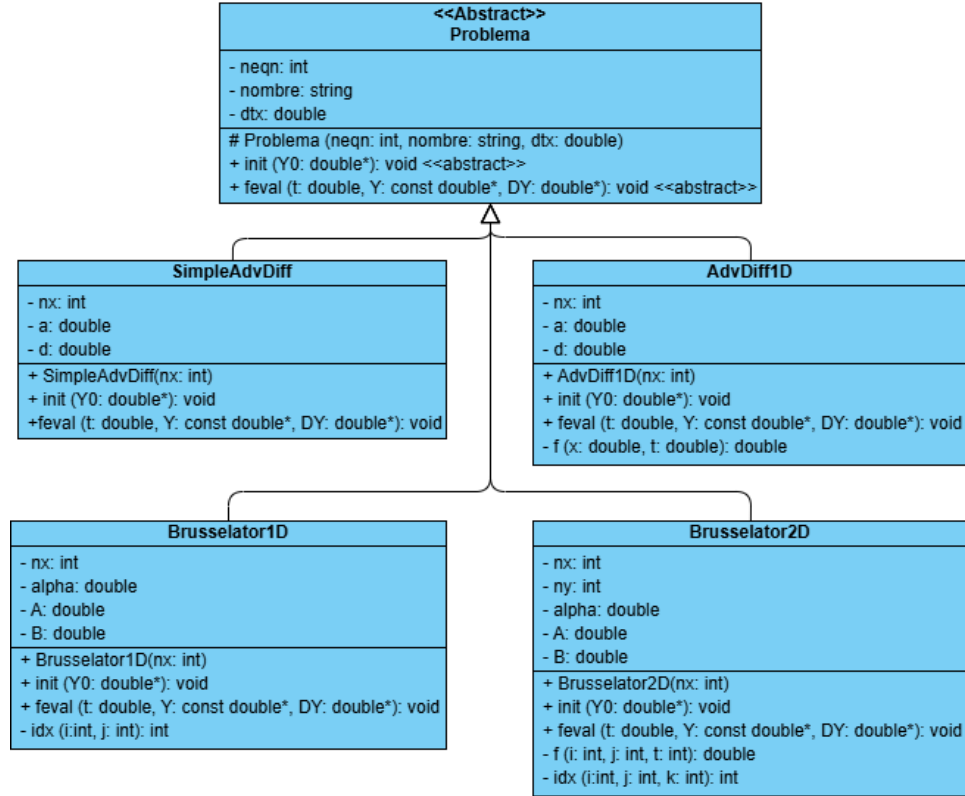


Figura 2: Diagrama de Clases de los problemas OMP

Como comentamos anteriormente, no nos centraremos en la resolución de los problemas *per se*, puesto que ésta se nos da ya hecha y corregida, sino que nuestros esfuerzos irán en implementar los métodos de resolución en ambas formas de paralelismo, y buscar la máxima eficiencia. Para facilitar la implementación y la cohesión del código, definimos una clase abstracta *Problema* de la cual heredarán los cuatro problemas.

El siguiente paso, por tanto, será realizar una implementación secuencial de los métodos, partiendo de los algoritmos que previamente hemos comentado, Runge-Kutta, Adams-Bashford, y Adamd-Bashford-Moulton, a saber, algoritmos 1, 2, y 3, respectivamente. El lenguaje elegido será C++, para luego poder añadir la parte de OpenMP o CUDA.

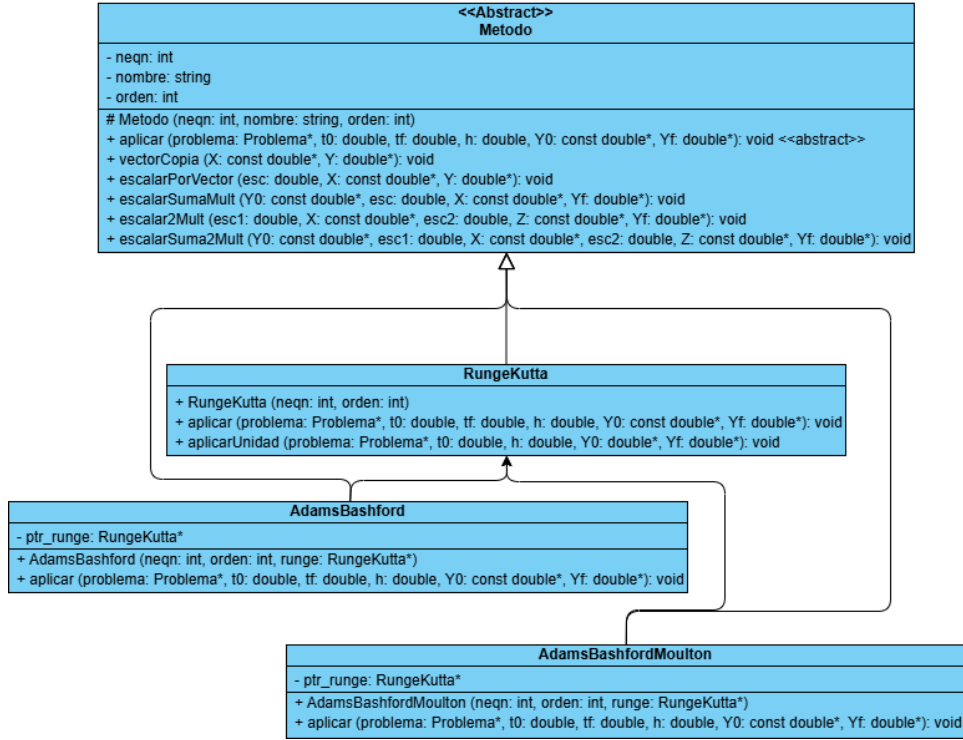


Figura 3: Diagrama de Clases de los métodos OMP

El planteamiento que se hace con los métodos es similar al de los problemas, una clase abstracta padre de la cual puedan heredar los métodos. Además, en la clase padre *Metodo* hemos implementado diversas funciones auxiliares que nos van a facilitar el trabajar con vectores. Con este diseño, a través de la herencia y el polimorfismo, podríamos aumentar el alcance de nuestro proyecto añadiendo nuevos métodos o más problemas sin dificultad ninguna.

Ahora que tenemos una primera maqueta funcional (aunque poco eficiente), podremos comenzar con el trabajo en paralelo. De cara a nuestro proyecto, lo primero que debemos tener en cuenta es que no podemos simplemente paralelizar el bucle principal con el que trabajamos (e.g. el bucle while en el algoritmo 2) porque cada iteración necesita de la anterior para calcular el siguiente paso. Debemos buscar por tanto otras formas de atacar el problema. En el caso de OpenMP hemos planteado dos formas, paralelismo por operaciones, y por componentes.

Paralelismo por Operaciones

En esta estrategia buscamos que cada función paralelice su propio trabajo. Eso quiere decir que cada operación vectorial, como el *feval()*, o *es-*

calarPorVector, tiene su propio `#pragma omp parallel`, crea un equipo de hilos, reparte el trabajo, y lo sincroniza.

De forma simplificada, veamos un ejemplo de Adams-Bashford 4:

Algoritmo 4 Implementación de AB4 con Paralelismo por Operaciones

```

 $Y_{n0} \leftarrow Y_0$ 
 $Y_n \leftarrow \text{arranqueRK4}(Y_{n-1})$   $\triangleright n = \{1, 2, 3\}$ 
 $F_n \leftarrow \text{feval}(t_n, Y_n)$   $\triangleright n = \{0, 1, 2, 3\}$ 
for  $t_n = t_0 + 3h; t_n < t_f; t_n += h$  do
     $\text{escalar2Mult}(37.0, F_{n1}, -9.0, F_{n0}, Y_{aux})$ 
     $\text{escalarSuma2Mult}(Y_{aux}, 55.0, F_{n3}, -59.0, F_{n2}, Y_{aux})$ 
     $\text{escalarSumaMult}(Y_{n3}, h/24.0, Y_{aux}, Y_{n4})$ 
     $F_{n4} \leftarrow \text{feval}(t_n + h, Y_{n4})$ 
     $\text{swap}(Y_{n4}, Y_{n3})$ 
     $\text{swap}(F_n, F_{n-1})$   $\triangleright n = \{1, 2, 3, 4\}$ 
end for
 $Y_f \leftarrow Y_{n3}$ 

```

Como podemos ver, para hacer el cálculo propio de Adams-Bashford de orden 4 (ecuación 15) la hemos dividido en 3 partes utilizando las funciones auxiliares de la clase abstracta *Metodo* de las que ya disponíamos. Dentro de cada una de ellas se creará el grupo de hilos, se repartirá el trabajo (cada hilo tendrá un "trozo" de vector sobre el que trabajar), y una vez finalicen se sincronizará para retornar a la función principal. Por ejemplo, *vectorCopia*():

Algoritmo 5 Implementación de Función Auxiliar *vectorCopia*

```

#pragma omp parallel for schedule(static)
for  $i=0; i < neqn; ++i$  do
     $Y[i] \leftarrow X[i]$ 
end for

```

Esta estrategia trae consigo ventajas como la modularidad, ya que cada operación vectorial es independiente y autocontenida, lo cual facilita la depuración y la legibilidad del código.

El principal punto negativo de esta estrategia es que trae consigo un *overhead* alto, ya que cada función tiene una barrera implícita al acabar un *parallel for*, en cada iteración del bucle hay varias llamadas a funciones, y el bucle itera un elevado número de veces. Esto equivale a más paradas de las que serían deseadas, además de que los hilos entran y salen de regiones paralelas constantemente.

Paralelismo por Componentes

Frente al paralelismo de operaciones, tenemos el paralelismo por componentes (o por elementos), en el que sólo tenemos una gran región *parallel* dentro de la cual ocurre todo el paralelismo, y cada hebra calcula su parte de los vectores.

Esto quiere decir que se creará un único equipo de hilos al entrar en el *parallel*, y todos los hilos permanecerán activos e iterarán en el bucle, paso a paso. Dentro del bucle iterativo, habrá diversos *#pragma omp for* que repartirán los índices entre los hilos. Esto además nos permitirá reducir las barreras de sincronización. Para poder comparar con la anterior implementación 4, describimos el mismo algoritmo (15) pero con esta estrategia:

Algoritmo 6 Implementación de AB4 con Paralelismo por Componentes

```

 $Y_{n0} \leftarrow Y_0$ 
 $Y_n \leftarrow \text{arranqueRK4}(Y_{n-1})$   $\triangleright n = \{1, 2, 3\}$ 
#pragma omp parallel for schedule(static)
for  $i=0; i < neqn; ++i$  do
     $F_n[i] \leftarrow \text{feval}_i(t_n, Y_n, i)$   $\triangleright n = \{0, 1, 2, 3\}$ 
end for
#pragma omp parallel
for  $t_n=t_0+3h; t_n < t_f; t_n+=h$  do
    #pragma omp for schedule(static)
    for  $i=0; i < neqn; ++i$  do
         $Y_4[i] = Y_3[i] + h/24 * (55F_3[i] - 59F_2[i] + 37F_1[i] - 9F_0[i])$ 
    end for
    #pragma omp for schedule(static)
    for  $i=0; i < neqn; ++i$  do
         $F_4[i] \leftarrow \text{feval}_i(t_n + h, Y_4, i)$ 
    end for
    #pragma omp single
    swap( $Y_{n4}, Y_{n3}$ )
    swap( $F_n, F_{n-1}$ )  $\triangleright n = \{1, 2, 3, 4\}$ 
end for
 $Y_f \leftarrow Y_{n3}$ 

```

La idea principal detrás de esta estrategia es reducir el *overhead* provocado por tener demasiadas barreras de sincronización, y mejorar el uso de la caché. Para mejorar la implementación hacemos uso de directivas como *schedule (static)*, con la que indicamos cómo deben repartirse las iteraciones entre los hilos. En este caso, se divide el rango de iteraciones en bloques contiguos y de tamaño fijo, y se asigna cada bloque a un hilo antes de empezar la ejecución del bucle. Esto es práctico cuando sabemos de antemano que todas las iteraciones van a costar lo mismo, y no va haber ramas pesadas o

irregulares.

Algunas de las desventajas que tendremos es que el código es más complejo, pues no se puede reutilizar el *feval()* secuencial (que sí lo aprovechábamos en el paralelismo por operaciones), sino que hay que hacer una versión específica para los componentes. De igual manera, no se pueden utilizar las funciones auxiliares, porque están diseñadas para tratar los vectores completos, no elementos individuales.

Mediciones

Ante la pregunta de qué estrategia es mejor, la respuesta sería, como casi siempre en informática, que depende.

En un primer planteamiento, parece que el paralelismo por componentes debería ser mejor. Reduce la creación y destrucción repetitiva de las hebras, y reduce las barreras de sincronización.

Un matiz importante que debemos hacer es que, aunque hablemos de creación y destrucción de hilos iterativa, no siempre ha de ocurrir así. Es cierto que, de forma lógica, el modelo *fork-join* de OpenMP describe de esa forma el comportamiento de los hilos; se crean, se usan, se destruyen. [Jin et al., 2024] Sin embargo, esto no siempre ocurre así (ya que sería muy costoso).

Existe un concepto llamado *thread pooling*, que describe que el entorno de ejecución podrá devolver los hilos a la *pool* en lugar de destruirlos, y así ahorrarse crearlos posteriormente. [Barlas, 2023] Esto eliminaría la principal desventaja del paralelismo por operaciones.

De forma análoga, el *runtime* no siempre mantiene los hilos activos (como pudiera parecer), aunque no haya finalizado el bloque *parallel*, ya que si el trabajo dentro de cada bloque *omp for* es pequeño, el compilador puede decidir dormir los hilos para ahorrar energía. Esto puede provocar pérdida de tiempo al tener que despertar los hilos, y latencias de sincronización más altas.

Otro dato a tener en cuenta, es que en el planteamiento de nuestros problemas, las llamadas a *feval_i()* son muy baratas, y el trabajo por iteración es pequeño, mientras que *feval()* es más pesado, y en cierta forma se amortiza el *overhead* al hacerse una única llamada en lugar de *neqn* llamadas.

Podemos comenzar, por ejemplo, comparando los tiempos al resolver el problema *Brusselator1D* con 100.000 ecuaciones.

Solver	Hebras	Estrategia	T (s)
Runge-Kutta	1	OMP Componentes	2501.84
		OMP Operaciones	1544.87
	4	OMP Componentes	670.504
		OMP Operaciones	371.93
	16	OMP Componentes	197.065
		OMP Operaciones	180.327
Adams-Bashford	1	OMP Componentes	707.634
		OMP Operaciones	556.051
	4	OMP Componentes	179.559
		OMP Operaciones	130.993
	16	OMP Componentes	56.5174
		OMP Operaciones	66.1485
Adams-Bashford-Moulton	1	OMP Componentes	2914.04
		OMP Operaciones	1772.73
	4	OMP Componentes	785.555
		OMP Operaciones	438.018
	16	OMP Componentes	237.236
		OMP Operaciones	219.918

Tabla 2: Tiempos de Brusselator1D con 100000 ecuaciones

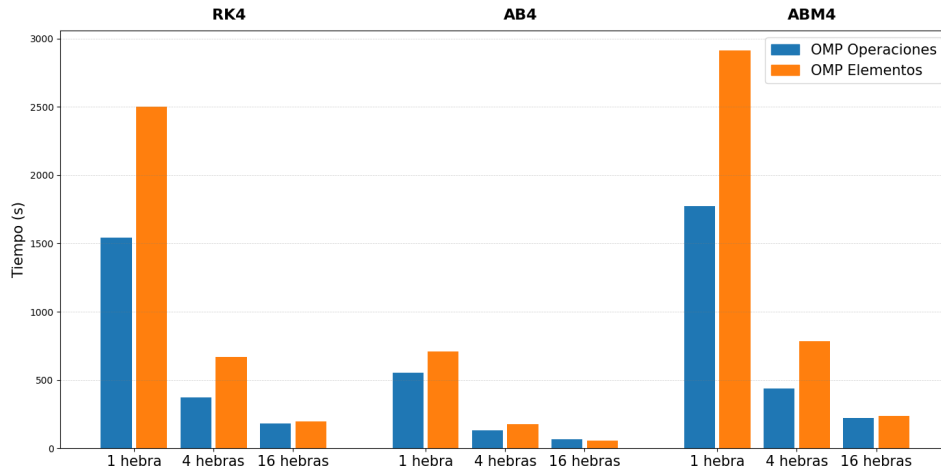


Figura 4: Brusselator1D 100.000 ecuaciones

Como podemos ver, hay una mejora sustancial del tiempo al pasar de 1 a 4 y a 16 hebras, con ambas estrategias de paralelismo, y con los 3 métodos numéricos. Además, se aprecia que la resolución con Adams-Bashford es más rápida que Runge-Kutta, ya que es más directa y no requiere de los

pasos intermedios (K1, K2, K3, K4), y también es mucho más veloz que el predictor-corrector Adams-Bashford-Moulton, ya que éste último llama a Adams-Bashford dentro de su bucle de ejecución. Además, podemos ver que para este tipo y tamaño de problema, el paralelismo por operaciones es más eficiente para 4 hebras, pero ambos se equiparan cuando se alcanzan las 16.

Podemos probar por ejemplo, con el problema más complejo (*Brusselator2D*), y el método que más tarda (*ABM4*), y podemos comparar la evolución del tiempo conforme aumenta el tamaño del problema, a saber, con un número de ecuaciones igual a 45000, 80000 y 125000.

neqn	Hebras	Estrategia	T (s)	Speedup	Eficiencia
45000	1	Operaciones	976.061	1	100
		Componentes	2151.79		
	4	Operaciones	263.081	3.710115896	92.7528974
		Componentes	569.693	3.777104511	94.4276128
	16	Operaciones	180.662	5.402691213	33.7668201
		Componentes	206.744	10.40799249	65.0499531
80000	1	Operaciones	1825.86	1	100
		Componentes	3970.37		
	4	Operaciones	466.282	3.915784868	97.8946217
		Componentes	1049.35	3.78364702	94.5911755
	16	Operaciones	214.67	8.505426934	53.1589183
		Componentes	331.809	11.96582974	74.7864359
125000	1	Operaciones	2692.79	1	100
		Componentes	5982.94		
	4	Operaciones	675.123	3.988591708	99.7147927
		Componentes	1593.61	3.754331361	93.8582840
	16	Operaciones	264.402	10.18445398	63.6528373
		Componentes	445.081	13.44236218	84.0147636

Tabla 3: Brusselator2D con ABM4

Para calcular el speedup y la eficiencia, hemos comparado los valores de mismo tamaño y estrategia, para que la comparación sea más justa. Más allá de lo obvio, que conforme aumenta el tamaño del problema, aumentan los tiempos, es cuanto menos destacable la variación en los speedups que se alcanzan en función del aumento de las hebras, en una estrategia y otra.

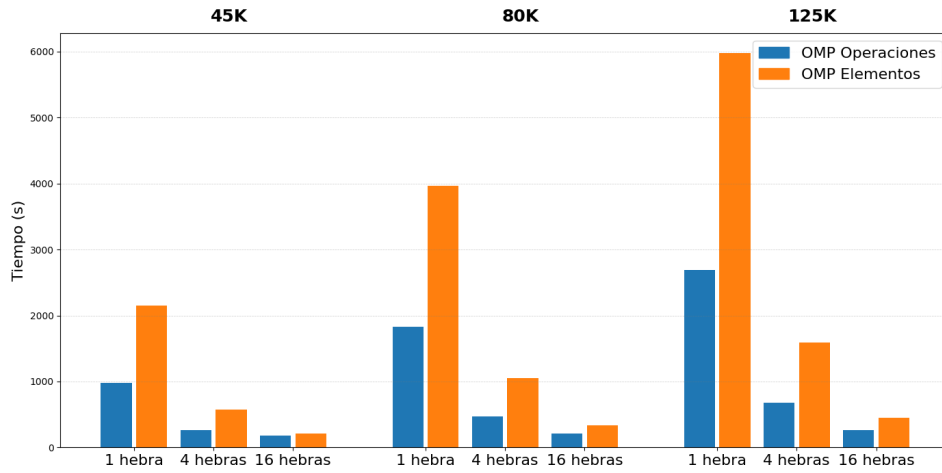


Figura 5: ABM4 resolviendo Brusselator2D

Partiendo de los mismos datos, pero tomando ahora solo el speedup y la eficiencia, y quedándonos con paralelismo de operaciones, tendríamos estas métricas:

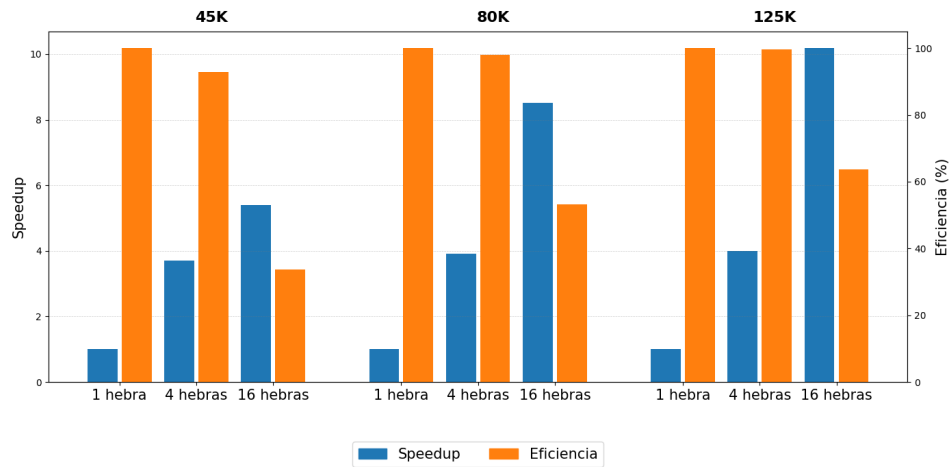


Figura 6: Speedup y Eficiencia - ABM4 Brusselator2D

Tenemos la barras del speedup en azul y alineadas con el eje Y izquierdo, y en naranja y con el eje Y derecho la eficiencia en valores porcentuales. Obviamente, para 1 hebra siempre tendremos un speedup de 1, y una eficiencia de 100 %.

Más allá de eso, podemos ver varias ideas en juego a la vez. Si nos centramos en cualquiera de los 3 tamaños, podemos ver la Ley de Amdahl [Amdahl, 1967] en funcionamiento; con un mismo tamaño de problema, conforme crece el número de hilos, aumenta el speedup pero disminuye la eficiencia, ya

que siempre queda una parte secuencial que no se puede paralelizar.

Análogo a esto, si nos fijamos ahora en las columnas de la eficiencia, podemos ver que, para un mismo número de hebras, conforme aumenta el tamaño de problema, la eficiencia lo hace con él, pasando de un 33 %, a un 53 %, y a un 63 %, cuando observamos los datos relativos a las 16 hebras por ejemplo. Esto coincide con la ley de Gustafson [[Gustafson, 1988](#)], ya que al aumentar el tamaño del problema, crece la parte del programa que es paralelizable, y por lo tanto la eficiencia aumenta también.

CUDA

CUDA, siglas de *Compute Unified Device Architecture* es una plataforma de computación paralela y un conjunto de APIs desarrollado por NVIDIA para ejecutar código general en sus GPUs. Incluye, entre otros, un modelo de programación a partir de C y C++; un compilador (*nvcc*) que genera código tanto para CPU como para GPU; un entorno de ejecución y una API para gestionar memoria, lanzamientos de kernels, streams; y librerías construidas sobre el propio modelo. [Kirk and Hwu, 2010] La idea principal es permitir que el programador trate a la GPU como un dispositivo de cómputo masivamente paralelo.

A modo de referencia, los primeros trabajos en los que se utiliza una GPU para cómputo general aparecen a principios de los 2000, en 2004 se crea internamente CUDA, y se lanza oficialmente en 2007. Desde entonces, la evolución de CUDA ha ido ligada a la evolución propia de las GPUs de NVIDIA, manteniendo el modelo *SIMT* y añadiendo unidades especializadas como *Tensor Cores*, *RT Cores*, o el mecanismo del *Transformer Engine*. [NVIDIA, 2026]

CUDA implementa un modelo SIMT (Single Instruction, Multiple Threads), en el cual el programador define un kernel (función `__global__`) que se ejecuta en muchos hilos. Los hilos se agrupan en bloques, y los bloques en una grid, y cada bloque se ejecuta en un *Streaming Multiprocessor*, dentro del cual los hilos se agrupan en warps de 32 hilos. [Kirk and Hwu, 2010] El warp es por tanto la unidad de ejecución con la que trabajamos.

Cuando se trabaja con CUDA se ha de tener en mente que se puede acceder a varias memorias diferentes, cada una con su propia latencia y ancho de banda, y entre ellas nos encontramos a los registros, que son la memoria de los hilos individuales, y la más rápida de todas; pasando por la memoria compartida (`__shared__`), que usan los bloques; la memoria constante (`__constant__`), que es de solo lectura, y va cacheada; y la memoria global, que se aloja en la VRAM de la GPU. [Farber, 2011]

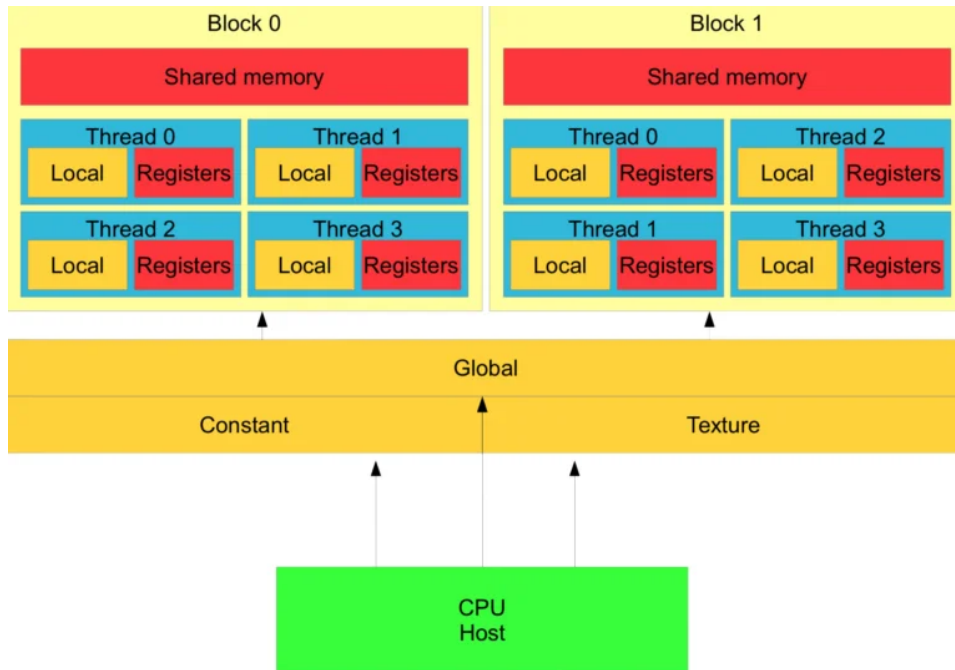


Figura 7: Jerarquía de memorias en CUDA [Beard, 2020]

El rendimiento de la memoria dependerá del buen hacer del programador, que será el encargado de buscar la coalescencia en los accesos a memoria global, del uso del *shared memory* para reutilizar datos, de que evite la divergencia de los warps, y de que se encargue de controlar los warps activos.

La coalescencia ocurre cuando los 32 hilos de un warp acceden a direcciones de memoria que caen dentro de un mismo segmento continuo. [Kirk and Hwu, 2010] Esto permite que la GPU atienda todos esos accesos con una sola transacción de memoria, lo cual implica una mayor eficiencia en el kernel.

Por otro lado, la divergencia de warps ocurre cuando los hilos de un mismo warp toman caminos distintos en una rama condicional. Como el warp ejecuta las instrucciones en modo SIMT, si los hilos divergen, el hardware debe serializar las rutas. [Farber, 2011] Esto quiere decir que si los hilos se encuentran frente a un bloque if/else, lo que va a pasar es que el bloque if será ejecutado por los hilos que cumplan la condición, y el resto de hilos se quedarán en stand-by. A continuación, el resto de los hilos ejecutará el bloque else mientras los primeros se quedan esperando, y luego se combinarán los resultados.

Hay un matiz importante a tener en cuenta a la hora de hablar de implementaciones CUDA, y es que esto que veremos aquí serán kernels, y no métodos de clase como anteriormente. Una forma de verlo es que tendremos clases (como *Metodo*), y éstas tendrán sus métodos que accederemos de for-

ma normal con un objeto de la clase, y por otro lado tendremos los kernel, que podrán ser llamados o no desde los métodos de las clases, y será donde ocurra el paralelismo de CUDA.

Aunque tengamos los kernel implementados en los archivos `.cu` de las clases, formalmente no son parte de ellas, son métodos globales que pueden ser llamados desde cualquier parte del código. Volviendo ahora a la implementación del kernel auxiliar que tenemos, hay varios conceptos a tener en cuenta.

Como antes mencionamos, los kernel CUDA funcionan con grids, bloques, y hebras, y estos elementos interaccionan entre sí. Cuando llamamos a un kernel se crea una grid, que no es más que un conjunto de bloques con un tamaño predeterminado. Esta grid podrá tener una, dos, o tres dimensiones, y en cualquiera de los casos contendrá un número determinado de bloques.

Estos bloques a su vez también tendrán de una a tres dimensiones y contendrán a las hebras, que serán con las que nosotros trabajaremos, indicando a cada hebra qué hacer dentro de los kernel. Si bien es cierto que aquí se aplicarán principalmente por defecto grids y bloques unidimensionales, salvo cuando se indique lo contrario.

Aquí podemos ver un ejemplo ilustrativo en el que un kernel particular es llamado y crea una grid bidimensional de bloques, y a su vez los bloques por dentro son tridimensionales.

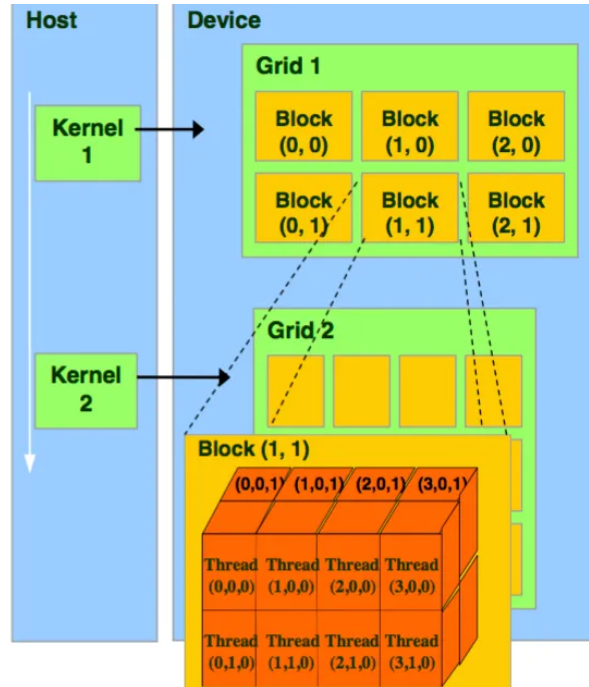


Figura 8: Grids y bloques CUDA [Casas, 2024]

Como primer ejemplo para ilustrar el modelo, podemos tomar una función auxiliar de la clase *Metodo* como la que ya vimos implementada en OpenMP (algoritmo 5). En este caso veremos el kernel *escalarPorVector()*.

Algoritmo 7 Kernel de *escalarPorVector*

```

 $i \leftarrow \text{blockDim.x} \cdot \text{blockIdx.x} + \text{threadIdx.x}$ 
if  $i < N$  then
     $Y[i] \leftarrow Y[i] + X[i] * \text{esc}$ 
end if

```

Por un lado, como podemos ver en la primera línea, se calcula un valor i que es igual a una multiplicación y una suma. Identificando a esos 3 elementos tenemos primero a *blockDim.x*, que es un valor igual al tamaño de la dimensión X del bloque al que pertenezca cada thread particular. Luego, *blockIdx.x* designa al bloque individual que contenga cada thread que ejecute el kernel, y de igual manera *threadIdx.x* será el identificador individual de cada thread dentro de su bloque. Combinando los 3 generamos un identificador único para cada uno de los threads que genere el grid.

Por otro lado, se hace la comprobación de que el identificador del thread es menor que el tamaño de los vectores para evitar posibles accesos erróneos a memoria. Esto ha de hacerse ya que, si tenemos un tamaño de problema N , se generará un grid con tantos bloques de tamaño *blockDim.x* como para que el total de hebras sea siempre mayor o igual a N . Dicho de otra forma, con un tamaño de bloque de 256, y un tamaño de problema de 1000, se generaría una grid con 4 bloques (y por tanto 1024 hebras), y las últimas 24 no trabajarían.

Implementación

Comenzaremos implementando los problemas. La estructura que tendremos será similar a la de OpenMP (figura 2) con un par de cambios que nacen de la naturaleza de CUDA. En todas las clases, además de las variables relativas al tamaño del problema (*neqn* y *nx*), también deberemos controlar las variables *grid* y *block*, que serán necesarias para el kernel.

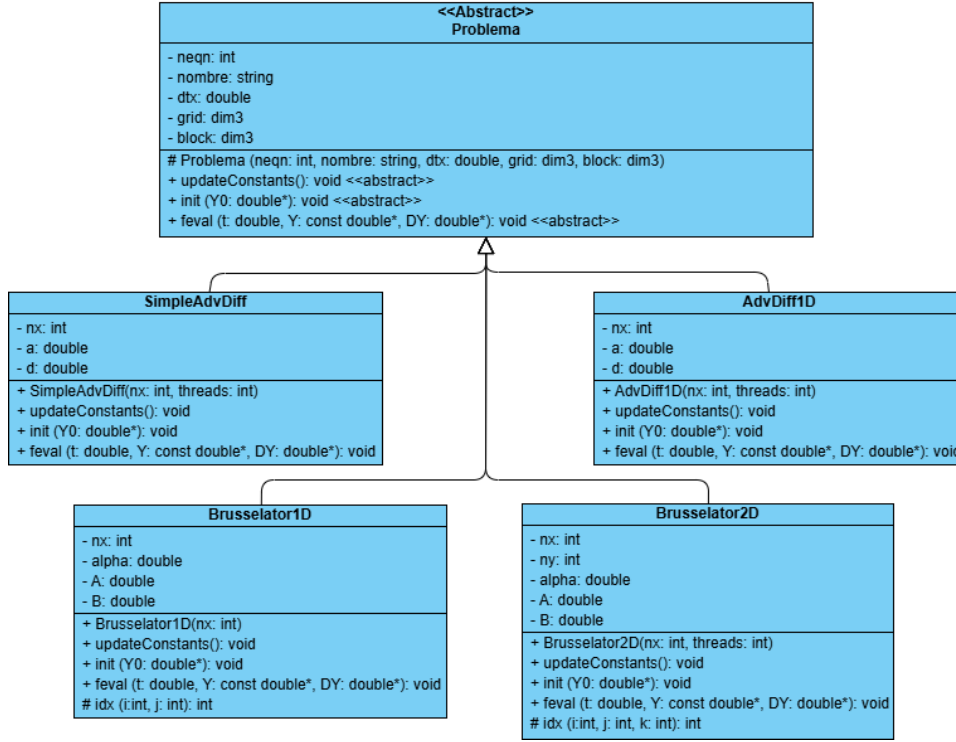


Figura 9: Diagrama de Clases CUDA de los problemas

Además, la función *feval()* sólo actuará como llamada al kernel, no tendrá ninguna computación dentro. Usamos este sistema porque, como mencionamos, los kernel no son funciones miembro y no pueden acceder a los atributos de las clases, así que la función *feval()* actúa de intermediaria entre los métodos y los kernel de los problemas. Además, se ha eliminado cualquier función auxiliar que se llamase internamente desde *feval()* ya que los kernel no pueden llamar a funciones que sean parte del host, métodos virtuales, o funciones miembro que utilicen a la CPU.

Algoritmo 8 Implementación de *feval* CUDA

```
kernel_problema<<<grid,block>>>(t, Y, DY);
```

A través de la herencia, las clases método podrán llamar a *feval()*, y que sea el propio de cada problema el que llame a su kernel con la configuración de grids y bloques correcta para el funcionamiento las herramientas CUDA.

Hay otra función que no teníamos en las implementaciones de OpenMP y es *updateConstants()*. El planteamiento es simple, tenemos el inconveniente de que los kernel no pueden acceder a métodos miembro de los objetos problema, como a los *getters*, y por tanto *feval()* debería pasar como argumento al kernel todas las variables del problema que necesite (como *nx*, o *B*).

Esto no sería una solución ideal porque, además de hacer poco legible al kernel, aunque no afectaría al rendimiento de la ejecución, sí lo haría a la eficiencia del lanzamiento del kernel. Esto se daría porque cada vez que se lanza un kernel, que en nuestro caso además los kernel *feval()* se lanzan muchas veces, los argumentos del mismo han de ser empaquetados por el *runtime* y copiados a un área de parámetros.

La solución que proponemos a este contratiempo es utilizar la memoria constante. [Farber, 2011] En cada header *.h* de los problemas se implementa una estructura que contendrá los parámetros que utilizará el kernel asociado a ese problema.

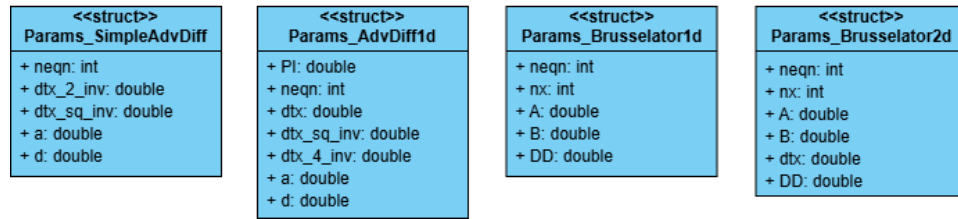


Figura 10: Diagrama de Clases de los structs

Posteriormente en el source *.cu* se declara un objeto de esa estructura.

Algoritmo 9 Declaración del struct constante

```
extern __constant__ Params_br1d cte_br1d;
```

Más tarde, los métodos de resolución desde su *aplicar()* podrán llamar a esta función miembro del problema, y ella se encargará de actualizar los valores y de transferirlos a memoria constante para que pueda ser utilizada por el kernel con una gran eficiencia.

Algoritmo 10 Función updateConstants() de Brusselator1D

```
Crear objeto aux de tipo Params_br1d
aux.neqn ← get_neqn()
aux.nx ← get_nx()
aux.A ← get_A()
aux.B ← get_B()
aux.DD ← get_DD()
cudaMemcpyToSymbol(cte_br1d, &aux, sizeof(Params_br1d))
```

Ahora que tenemos todas las partes que nos faltaban podemos pasar a los kernels. En el planteamiento que hemos hecho de las clases, cada clase problema tendrá un único kernel encargado de hacer las transformaciones necesarias (como hacía *feval* en OpenMP). Veamos la versión resumida de uno:

Algoritmo 11 Kernel feval de AdvDiff1D

```
 $thread \leftarrow blockDim.x \cdot blockIdx.x + threadIdx.x$   
 $ult \leftarrow cte\_adv1.n eqn - 1$   
if  $thread < cte\_adv1.n eqn$  then ▷ Check de hebras  
   $i\_ant \leftarrow (thread == 0) ? ult : (thread - 1)$  ▷ Vecinos y frontera  
   $i\_pst \leftarrow (thread == ult) ? 0 : (thread + 1)$   
   $val\_ant \leftarrow Y[i\_ant]$  ▷ Obtenemos valores  
   $valor \leftarrow Y[thread]$   
   $val\_pst \leftarrow Y[i\_pst]$   
   $DY[thread] \leftarrow$  Cálculo usando  $val\_ant$ ,  $valor$ ,  $val\_pst$ ,  $thread$ ,  $t$ ,  $cte\_adv1$   
end if
```

En el ejemplo podemos diferenciar varias partes. Primero declaramos algunos valores que se usarán adelante, como el número de hilo único de cada thread, o el identificador común del último, que obtenemos de la memoria constante. Posteriormente, tras la comprobación de que sólo ejecuten tantos hilos como elementos del vector, obtenemos los identificadores de los vecinos. Se usan operadores ternarios para corregir los casos frontera en los que el índice vecino se salga de lo acotado; en el thread primero y en el último.

Finalmente, ahora que disponemos de los identificadores, obtenemos del vector alojado en memoria global el valor que corresponde tanto al hilo en cuestión, como a los vecinos. Y una vez que se tienen todos estos valores se procede al cálculo del nuevo valor que tendrá $DY[thread]$ y se guarda en el vector salida. Como hemos mencionado, todos los hilos del kernel lo ejecutarán en paralelo.

Teniendo cubierta la parte de los problemas, pasemos ahora a los métodos. De forma análoga a la implementación de los problemas, hay varios cambios que deben ser mencionados. Seguimos teniendo una clase abstracta padre *Metodo* que sirve de template a las hijas. En esta implementación, las funciones auxiliares que tenía la clase ahora son kernels con la misma funcionalidad, como podemos ver en el algoritmo 7.

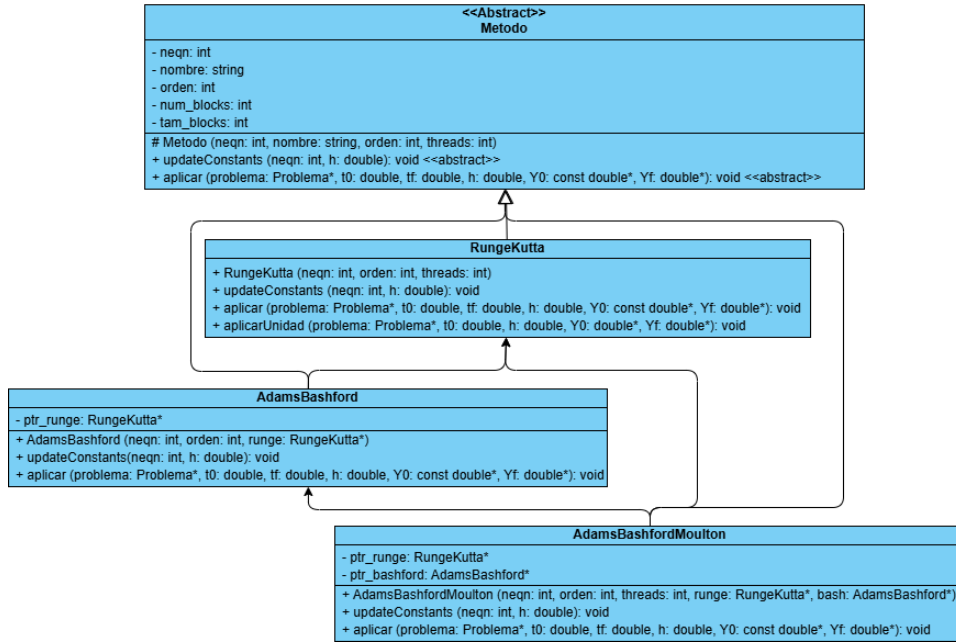


Figura 11: Diagrama de Clases de los Métodos CUDA

De igual manera, tanto la clase padre como las tres hijas tienen un *struct* asociado alojado en memoria constante, y un método propio *updateConstants()* encargado de actualizar ese *struct* constante antes de hacer uso de sus kernels.

Pasamos ahora al método *aplicar()*, la implementación en los 3 métodos es similar, cambiando sólo los pasos que se siguen para pasar de Y_n a Y_{n+1} . Veamos por ejemplo, Runge-Kutta de orden 2.

Algoritmo 12 Aplicar Runge-Kutta en CUDA

```

Reservamos memoria para los vectores
updateConstants(neqn, h)                                ▷ Memoria constante
problema->updateConstants(neqn, h)
 $Y_n \leftarrow Y_0$ 
for  $t_n=t_0; t_n < t_f; t_n+=h$  do
     $K1 \leftarrow f_{eval}(t_n; Y_n)$ 
     $Y_{aux} \leftarrow kernel\_escalarSumaMult(Y_n, h/2, K1)$ 
     $K2 \leftarrow f_{eval}(t_n + h/2; Y_{aux})$ 
     $Y_n \leftarrow kernel\_sumatoriaRK2(K1, K2)$                 ▷ Kernel principal RK2
end for
 $Y_f \leftarrow Y_n$ 
  
```

Cabe destacar que las funciones *aplicar()* siempre incluyen los 4 órdenes mencionados, aunque por facilidad de comprensión del pseudocódigo sólo se

ilustra aquí uno de los órdenes. Además, los kernel nunca devuelven ningún dato, todo debe pasárseles como atributo.

Debido a la naturaleza de estos métodos numéricos, que utilizan los valores presentes para predecir los futuros, no podemos evitar el bucle for con el que se itera, y sólo podemos trabajar para paralelizar dentro de él, sabiendo que el vector con el que acaba una iteración será con el que empiece la siguiente.

Mediciones

Comenzaremos midiendo los tiempos de los 3 métodos de resolución frente al problema Brusselator2D con 125.000 ecuaciones, y comparando los resultados entre la ejecución secuencial, OpenMP con 16 hebras, y CUDA con un tamaño de bloque unidimensional de 256 threads. Además, obtendremos el speedup comparando con la medición secuencial.

Solver	Tipo	T (s)	Speedup
Runge Kutta	Secuencial	2531.01	1
	OMP	236.42	10.70556636
	CUDA	77.1447	32.80860513
Adams Bashford	Secuencial	847.87	1
	OMP	82.704	10.25186206
	CUDA	23.5544	35.99624699
Adams Bashford Moulton	Secuencial	2692.79	1
	OMP	264.402	10.18445398
	CUDA	86.2916	31.20570252

Tabla 4: Comparativa con Brusselator2D 125K entre tecnologías

Si bien sí podemos calcular el speedup, no tiene sentido calcular la eficiencia, ya que las hebras GPU y las hebras CPU no son comparables. En cualquier caso, podemos apreciar la similar ganancia en velocidad al pasar a CUDA con cualquiera de los métodos.

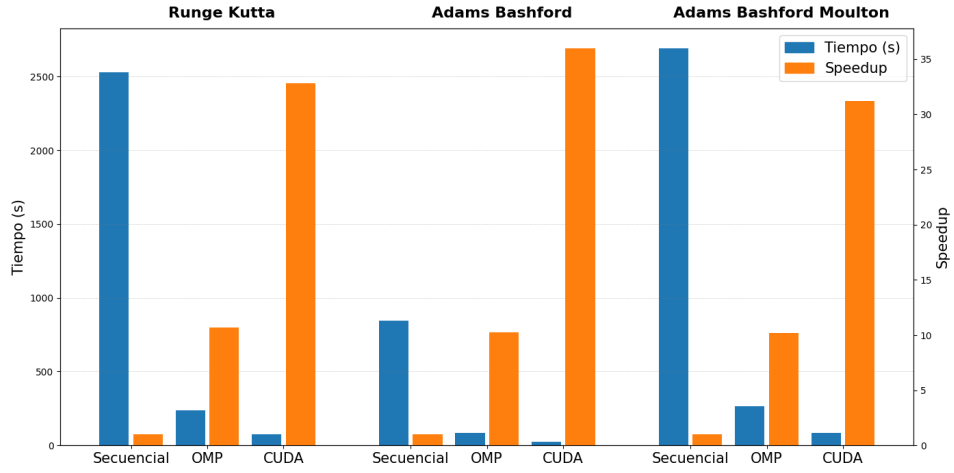


Figura 12: Comparativa con Brusselator2D 125K entre tecnologías

Si nos centramos ahora en el desempeño de las herramientas CUDA a través de varios tamaños de problema, por ejemplo, el método Runge-Kutta de orden 4 aplicado al problema Brusselator1D:

neqn	T (s)	ms/eq	neqn	T (s)	ms/eq	neqn	T (s)	ms/eq
100	18.96	189.6	40K	28.27	0.706	400K	195.79	0.489
200	18.84	94.21	60K	33.41	0.556	500K	246.38	0.492
400	19.16	47.91	80K	38.56	0.482	600K	291.95	0.486
1K	17.08	17.08	100K	43.40	0.434	700K	335.23	0.478
2K	17.09	8.548	150K	68.54	0.456	800K	379.27	0.474
4K	17.20	4.300	200K	89.16	0.445	900K	423.52	0.470
10K	17.43	1.743	250K	115.64	0.462	1M	469.07	0.469
20K	22.30	1.115	300K	141.37	0.471	1.5M	702.06	0.468
30K	23.58	0.786	350K	173.56	0.495	2M	927.77	0.463

Tabla 5: RK4 a través de tamaños de Brusselator1D en CUDA

O de forma gráfica, con el tiempo total en azul y en el eje Y logarítmico, y el tiempo por ecuación en naranja y el eje Y derecho logarítmico:

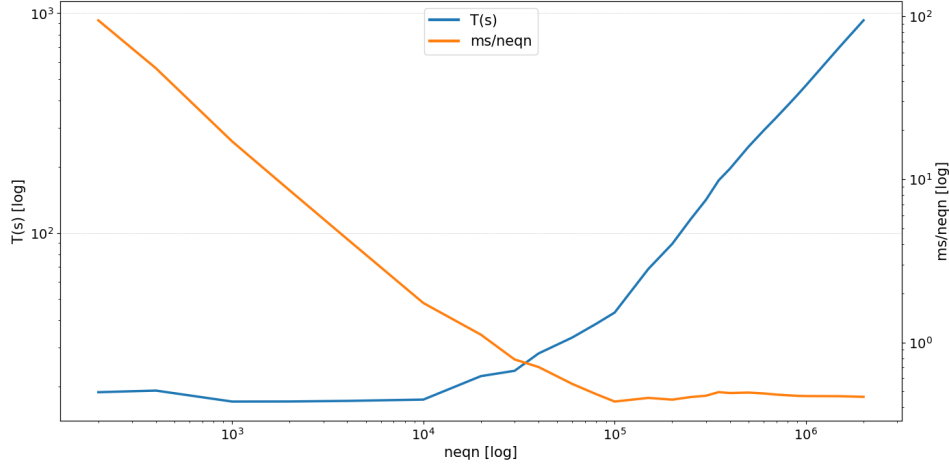


Figura 13: RK4 a través de tamaños de Brusselator1D en CUDA

Aquí podemos apreciar dos dinámicas distintas interactuando entre ellas. Por un lado, observando la línea del tiempo (azul) vemos como para tamaños pequeños, la parte secuencial del problema domina la ejecución, y el tiempo paralelo es casi cero, lo que implica que el total prácticamente no varía (o incluso disminuye) con respecto al aumento de tamaño hasta llegar a un tamaño en torno a 100K ecuaciones. A partir de esa cifra es cuando CUDA puede empezar a trabajar realmente, y el tiempo aumenta de forma lineal. Esto coincide con lo esperado.

Por otro lado, si observamos la curva naranja, de tiempo por ecuación (en milisegundos), vemos el efecto contrario. Conforme aumenta el tamaño del problema y CUDA puede trabajar con más ecuaciones, este cociente se reduce conforme aumenta el tamaño. Sin embargo, a partir de cierto punto se estabiliza, que será cuando se alcanza la eficiencia máxima.

Graphs

Tras ver el bucle de ejecución de la función miembro *aplicar()*, salta a la vista que se llama a uno o varios kernel de forma repetitiva y con los mismos datos, con los mismos vectores, y sólo varía la variable del tiempo. A raíz de esto se decide implementar una nueva versión de las clases métodos, que utilice los *Graph*.

Los CUDA Graphs son una representación estática de un conjunto de operaciones CUDA (kernels, copias, eventos) con sus dependencias. [Barlas, 2023] Se capturan una vez y se ejecutan muchas veces con mínimo overhead. Está formado por nodos que representan las operaciones realizadas, y edges que representan dependencias de ejecución.

El funcionamiento es relativamente simple, se capturan una secuencia de

operaciones (*cudaStreamCapture*), se analizan las dependencias y se optimiza el grafo, se instancia (*cudaGraphInstantiate*), y se ejecuta (*cudaGraphLaunch*) tantas veces como se desee. [NVIDIA, 2026]

Los graph CUDA presentan ciertas ventajas, como la reducción en el overhead del lanzamiento de kernels, la optimización de las dependencias, o la fusión interna de nodos. Algunas de sus limitaciones es que la estructura del grafo debe ser estática, o que no se pueden cambiar punteros capturados, ni introducir ramas dinámicas.

Para poder implementar los graph tuvimos que ampliar nuestro diagrama de clases, a saber, resumidamente:

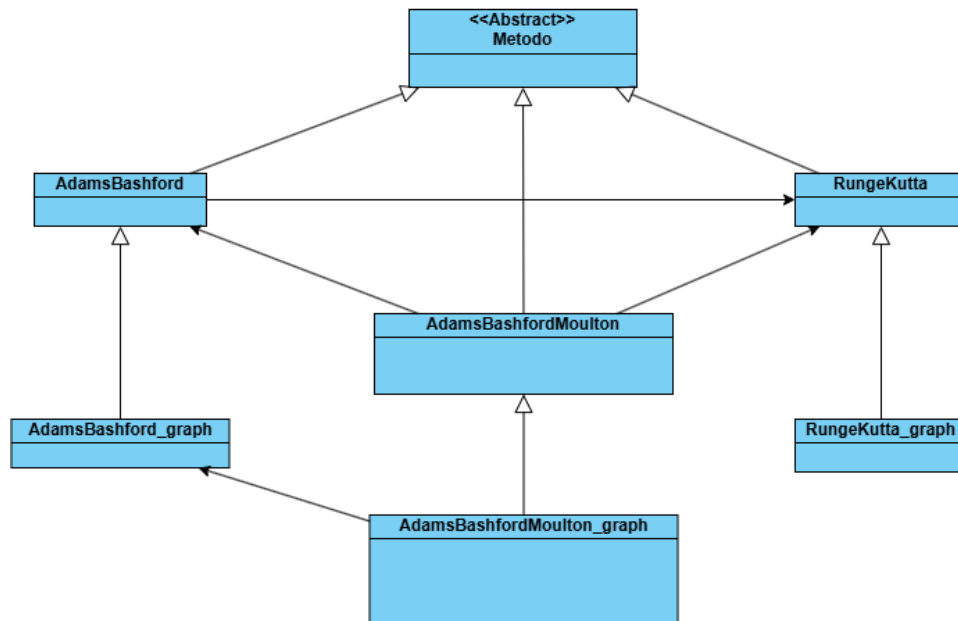


Figura 14: Dependencias entre las clases Graph

Se han creado nuevas clases "*problema_graph*" que heredan de las anteriores para reutilizar todos sus métodos salvo *aplicar()*, que lo sobrescriben. Dentro de él, capturan el grafo y lo lanzan las veces necesarias, pero con un detalle. Ya mencionamos que los kernel del bucle for eran iguales en cada iteración salvo por el tiempo (t_n) que sí cambiaba con cada iteración. Para tener esto en cuenta, cada vez que se va a lanzar el grafo se ha de modificar el objeto t_n a través de *cudaMemcpyToSymbolAsync* y la nueva función *get_t()*.

Por otro lado, también se modificaron los *feval()* de los problemas para adecuarlos a los Graph. En este caso, simplemente se sobrecargó el método *feval()*, para que en lugar del tiempo t_n recibiera el *cudaStream* en el que estuviera alojado el *Graph*. Estos métodos a su vez, en lugar de llamar al

kernel ya conocido, llamarán a un nuevo kernel también adaptado para los graph.

neqn	T(s) - CON	T(s) - SIN	Cociente
1250	15.6045	15.7724	1.010759717
5000	15.3411	15.7544	1.026940702
11250	15.9351	16.1561	1.013868755
20000	22.6578	22.6586	1.000035308
31250	23.8161	23.6049	0.991132049
45000	31.6381	30.7777	0.972804941
61250	39.9935	39.1357	0.978551515
80000	47.0315	43.5740	0.926485441
101250	69.7740	62.0139	0.888782354
125000	99.3000	86.2916	0.868998993
180000	147.217	120.547	0.818838857
320000	264.624	213.418	0.806495254
500000	392.972	313.984	0.798998402
720000	573.024	458.030	0.799320796
980000	772.954	615.165	0.795862367

Tabla 6: Comparación de tiempos de graph con ABM en Brusselator2D

Sin embargo, como podemos ver en los resultados de nuestras mediciones, la introducción de los *Graph* no solo no representó una mejora en los tiempos, sino que conforme aumentaban los tamaños, se estabilizaba en quedar un 20% más lenta que la versión sin *Graph*.

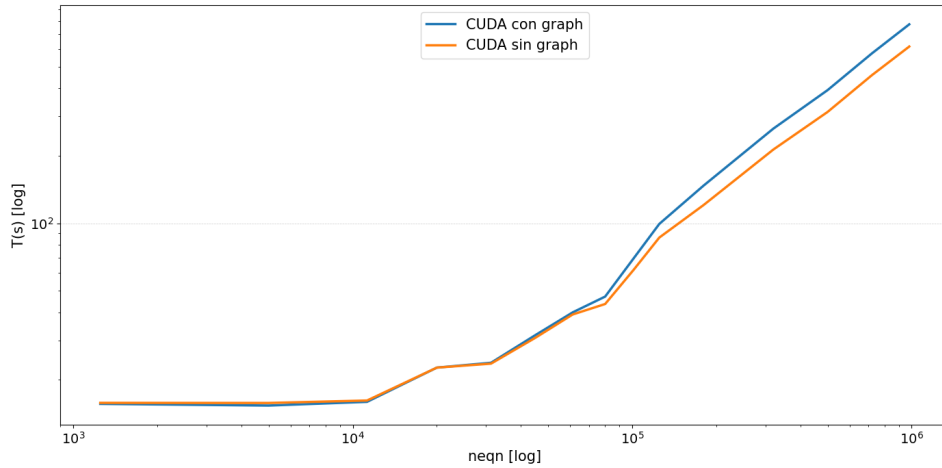


Figura 15: Comparación de tiempos de graph con ABM en Brusselator2D

Esto puede deberse a que el uso de CUDA Graphs sólo acelera la ejecu-

ción cuando el overhead de lanzamiento de kernels es significativo respecto al coste del kernel, como por ejemplo cuando los kernels son muy rápidos y el lanzamiento es costoso, pero como los nuestros son relativamente pesados (principalmente por sus múltiples accesos a memoria global), pues se da el caso de que el *Graph* cuesta más de lo que ahorra.

Shuffles y Grids multidimensionales

Para intentar minimizar los accesos a memoria dentro de lo posible, se hace uso de otra herramienta que CUDA pone a nuestra disposición, los *shuffle*. Los warp shuffle son instrucciones a nivel de warp que permiten que un hilo lea el valor de un registro de otro hilo del mismo warp.

Cada hilo llama a la misma función, pero puede recibir el valor de otro hilo del warp según el patrón especificado. A nivel de hardware, esto se implementa mediante una red de interconexión interna del warp (warp crossbar) que mueve datos entre registros sin tener que pasar por la memoria global, lo cual reduce los accesos, y por tanto el tiempo de ejecución del kernel.

Aunque el programador define a través del *blocksize* el tamaño de los bloques que forman el grid (por ejemplo, un tamaño habitual sería un bloque unidimensional de 256 hebras), a nivel de hardware CUDA se organiza en warps de 32 hilos consecutivos que se mueven al unísono, y es entre ellos donde ocurre la comunicación que permiten los *shuffle*.

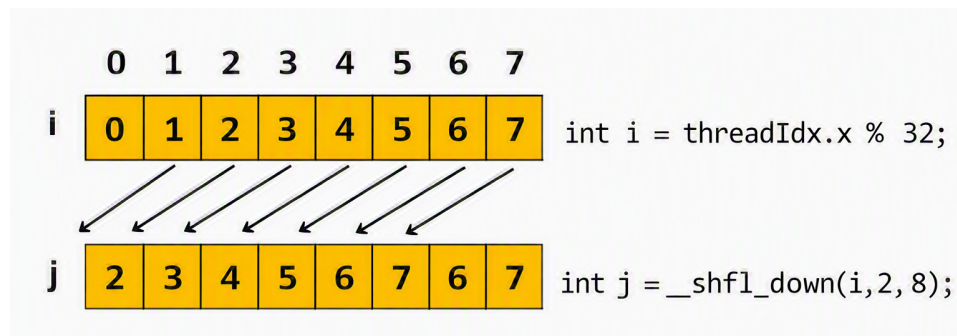


Figura 16: Ejemplo de shuffle down [Luitjens, 2014]

La imagen ilustra un ejemplo de shuffle. Los hilos crean una variable local (*i*), en este caso su propio identificador dentro del warp, y a través del shuffle podrán accederse entre ellos a esa variable. En este caso, cada hilo accede a esa variable *i* de los vecinos situados dos hilos por encima, y copia el contenido de la variable del vecino en una nueva propia (*j*).

Esto resulta útil en implementaciones como el algoritmo 11, en el que un mismo thread hace varios accesos a memoria global, y esto podríamos sustituirlo por un único acceso a memoria global, y varios shuffle entre hilos.

Algoritmo 13 Kernel feval de AdvDiff1D con shuffle

```
thread ← blockDim.x · blockIdx.x + threadIdx.x
ult ← cte_avd1.neqn - 1
lane ← threadIdx.x & 31                                ▷ Posicion en el warp
if thread < cte_avd1.neqn then
    mask = 0xFFFFFFFF
    valor ← Y[thread]                                ▷ Acceso a memoria
    val_ant ← __shfl_up_sync (mask, valor, 1)        ▷ Valores vecinos
    val_pst ← __shfl_down_sync (mask, valor, 1)
    if lane == 0 || thread == 0 then                ▷ Corrección de valores frontera
        val_ant ← (thread == 0) ? Y[ult] : Y[thread - 1]
    end if
    if lane == 31 || thread == ult then
        val_pst ← (thread == ult) ? Y[0] : Y[thread + 1]
    end if
    DY[thread] ← Cálculo usando val_ant, valor, val_pst, thread, t, cte_avd1
end if
```

En este ejemplo, en lugar de que cada hilo acceda 3 veces a memoria, acceden una sola, y es en los casos frontera en los que se hacen accesos extra puntuales.

Para lograr esta implementación se crean nuevas clases *problema_shuffle* que heredan de las clases estándar, sobrecargan la función miembro *feval*, e implementan su propia versión del kernel.

También se ha realizado otra nueva implementación aplicando otra innovación distinta de CUDA, los grid multidimensionales. Como pudimos ver en el algoritmo 8, a la hora de llamar a un kernel se le ha de indicar tanto las dimensiones de la grid como de los bloques que ésta contiene. Si bien se pueden utilizar enteros básicos (que CUDA interpreta como valores unidimensionales), también podemos indicarle a CUDA que vamos a trabajar con grids multidimensionales.

Para poner esto en práctica, se crea una clase hija llamada *problema_grid* que hereda de la clase estándar, y sobrecarga las función (*init()*), ya que la organización de los hilos será distinta, y la función *feval()*, que realizará otros accesos. En un problema como Brusselator1D, en el que cada elemento que lo compone tiene dos componentes, *u* y *v*, estas componentes pueden reinterpretarse como una segunda dimensión. Así se vería:

Algoritmo 14 Kernel feval de Brusselator1D con grid multidimensional

```
id_x ← blockDim.x · blockIdx.x + threadIdx.x           ▷ Coordenada X
id_y ← threadIdx.y                                     ▷ Coordenada Y
ult ← cte.nx − 1
lane ← threadIdx.x & 31                                ▷ Posicion en el warp
thread ← id_y · cte.nx + id_x
if id_x < cte.nx && id_y < 2 && thread < cte.neqn then
  valor ← Y[thread]
  mask = 0xFFFFFFFF
  C = (id_y == 0) ? cte.A : cte.B;
  val_izq ← __shfl_up_sync (mask, valor, 1)           ▷ Valores vecinos
  val_der ← __shfl_down_sync (mask, valor, 1)
  if lane == 0 || id_x == 0 then                       ▷ Corrección de valores frontera
    val_izq ← (id_x == 0) ? C : Y[thread − 1]
  end if
  if lane == 31 || id_x == ult then
    val_der ← (id_x == ult) ? C : Y[thread + 1]
  end if
  val_pareja ← (id_y == 0) ? Y[thread + cte.nx] : Y[thread − cte.nx]
  DY[thread] ← Cálculo usando val_izq, valor, val_der, thread, t, cte
end if
```

En la implementación original, los datos se almacenaban en bloques unidimensionales formando un Array of Structures.

$$[0_u, 0_v, 1_u, 1_v, \dots, (n-2)_u, (n-2)_v, (n-1)_u, (n-1)_v] \quad (22)$$

Mientras que en esta implementación cambiamos el paradigma a bloques bidimensionales que forman un Structure of Arrays.

$$[0_u, 1_u, \dots, (n-2)_u, (n-1)_u, 0_v, 1_v, \dots, (n-2)_v, (n-1)_v] \quad (23)$$

Sin embargo, yendo a las mediciones podemos ver que no ocurren cambios significativos con ninguna de las dos nuevas implementaciones frente a la anterior “estándar”.

neqn	T (s) de implementación			neqn	T (s) de implementación		
	Estándar	Shuffle	Grid		Estándar	Shuffle	Grid
200	4.25118	4.27395	4.40917	300000	49.0773	47.6186	47.1094
400	4.38250	4.47631	4.42845	400000	64.3164	64.0661	62.2658
1000	4.42428	4.48073	4.45638	500000	78.4998	79.2419	79.3901
2000	4.47256	4.49739	4.55980	600000	92.2143	93.8084	94.2763
4000	4.50782	4.55148	4.52020	700000	108.716	108.272	108.879
10000	4.64262	4.71993	4.68111	800000	123.306	122.247	122.502
20000	6.07753	6.19400	6.26689	900000	137.599	136.730	137.174
40000	7.65326	7.71357	7.82265	1000000	152.187	151.235	151.697
80000	10.92320	10.54850	10.69110	1200000	180.576	179.841	180.080
100000	13.19100	12.06130	12.23360	1400000	209.347	208.619	209.053
150000	23.07470	21.85070	22.02480	1600000	238.003	237.406	237.448
200000	30.91270	29.03480	29.69920	1800000	267.296	266.385	266.490
250000	41.32400	37.76580	38.72750	2000000	296.847	294.948	295.242

Tabla 7: Tiempos de AB4 en Brusselator1D estándar, Shuffle y Grid

Estos resultados concuerdan con lo esperado, teniendo en cuenta la naturaleza de los problemas con los que hemos trabajado y los cambios implementados. Si bien con otro tipo de problemas podrían ser beneficiosos cambios de ésta índole, en los nuestros las mejoras de tiempo de computación están limitadas por el ancho de banda de memoria global, la latencia de VRAM, y la coalescencia de accesos.

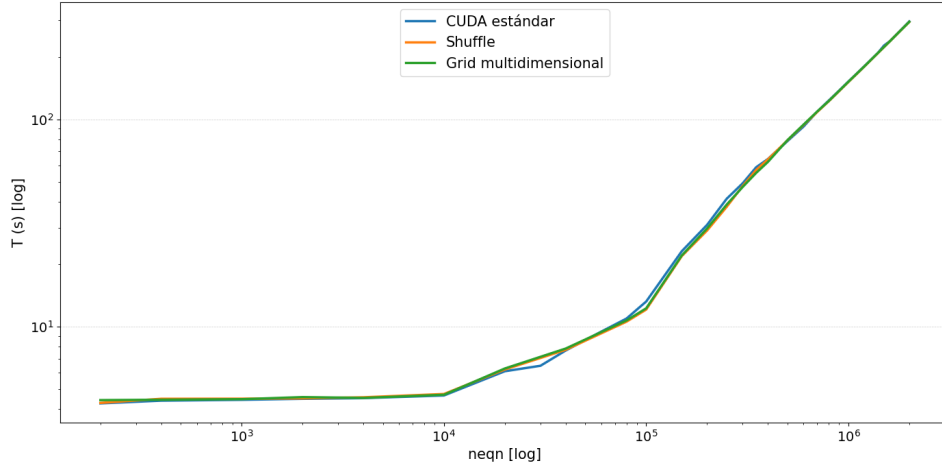


Figura 17: Tiempos de AB4 en Brusselator1D estándar, Shuffle y Grid

Bibliografía

- [Amdahl, 1967] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Spring Joint Computer Conference*. <https://dl.acm.org/doi/10.1145/1465482.1465560>.
- [ARB, 2024] ARB, O. (2024). Openmp 6.0 api syntax reference guide directives and constructs. <https://www.openmp.org/resources/refguides/>.
- [Ascher et al., 1995] Ascher, U. M., Ruuth, S. J., and Wetton, B. T. R. (1995). Implicit-explicit methods for time-dependent partial differential equations. *Society for Industrial and Applied Mathematics*, 32:797–823. <https://www.jstor.org/stable/2158449?origin=JSTOR-pdf>.
- [Barlas, 2023] Barlas, G. (2023). *Multicore and GPU Programming, An Integrated Approach*. Morgan Kaufman Publisher, second edition.
- [Beard, 2020] Beard, T. (2020). Cuda memory hierarchy. <http://thebeardsage.com/cuda-memory-hierarchy/>.
- [Calvo et al., 2001] Calvo, M. P., Frutos, J. D., and Novo, J. (2001). Linearly implicit runge-kutta methods for advection-reaction-diffusion equations. *Applied Numerical Mathematics*, 37:535–549. <https://bluehound2.circ.rochester.edu/astrobear/raw-attachment/blog/shuleli08202013/Calvo2001.pdf>.
- [Casas, 2024] Casas, A. (2024). Cuda tutorial – introduction to blocks and grids. <https://blog.damavis.com/en/cuda-tutorial-blocks-and-grids/>.
- [Farber, 2011] Farber, R. (2011). *CUDA Application Design and Development*. Morgan Kaufman.
- [Gallardo, 2018] Gallardo, J. M. (2018). *Análisis Numérico de Ecuaciones Diferenciales, Teoría y ejemplos con Python*. Universidad de Málaga.
- [Gustafson, 1988] Gustafson, J. (1988). Reevaluating amdahl’s law. *Communications of the ACM*, 31. <http://www.johngustafson.net/pubs/pub13/amdahl.pdf>.

- [Jin et al., 2024] Jin, H., Pophale, S., and Milfeld, K. (2024). *OpenMP Application Programming Interface Examples*. OpenMP Architecture Review Board, 6.0 edition. <https://www.openmp.org/wp-content/uploads/openmp-examples-6.0.pdf>.
- [Kirk and Hwu, 2010] Kirk, D. B. and Hwu, W. W. (2010). *Programming Massively Parallel Processors, A Hands-on Approach*. Morgan Kaufman.
- [Luitjens, 2014] Luitjens, J. (2014). Faster parallel reductions on kepler. <https://developer.nvidia.com/blog/faster-parallel-reductions-kepler/>.
- [Mantas, 2024] Mantas, J. M. (2024). Vssbdf: Variable stepsize semi-implicit backward differentiation formula solvers in c++. <https://github.com/jmmantas/VSSBDF>.
- [Mara’beh et al., 2024] Mara’beh, R. A., Mantas, J. M., González, P., and Spiteri, R. J. (2024). Performance comparison of variable-stepsize imex sbdf methods on advection-diffusion-reaction models. *Computers and Mathematics with Applications*.
- [Molero et al., 2007] Molero, M., Salvador, A., Menarguez, T., and Garmendia, L. (2007). *Análisis Matemático para Ingeniería*. Pearson Educación.
- [NVIDIA, 2026] NVIDIA (2026). *CUDA Programming Guide*. 13.3 edition. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>.
- [Schryen, 2024] Schryen, G. (2024). Speedup and efficiency of computational parallelization: A unifying approach and asymptotic analysis. *Journal of Parallel and Distributed Computing*, 187. <https://www.sciencedirect.com/science/article/pii/S0743731523002058>.
- [Segarra-Escandon, 2020] Segarra-Escandon, J. (2020). Metodos numericos, runge-kutta y adams bashford-moulton en mathematica. *Revista Ingeniería, Matemáticas y Ciencias de la Información*, 7:13–32. <https://ojs.urepublicana.edu.co/index.php/ingenieria/article/view/639/507>.
- [Zill, 2009] Zill, D. G. (2009). *Ecuaciones diferenciales con aplicaciones de modelado*. Brooks/Cole, Cengage Learning, novena edition.

Listado de Algoritmos

1.	Runge-Kutta de Orden 4	12
2.	Adams-Bashford general	14
3.	Adams-Bashford-Moulton general	15
4.	Implementación de AB4 con Paralelismo por Operaciones . .	20
5.	Implementación de Función Auxiliar vectorCopia	20
6.	Implementación de AB4 con Paralelismo por Componentes .	21
7.	Kernel de escalarPorVector	30
8.	Implementación de feval CUDA	31
9.	Declaración del struct constante	32
10.	Función updateConstants() de Brusselator1D	32
11.	Kernel feval de AdvDiff1D	33
12.	Aplicar Runge-Kutta en CUDA	34
13.	Kernel feval de AdvDiff1D con shuffle	41
14.	Kernel feval de Brusselator1D con grid multidimensional . . .	42

Listado de Figuras

1.	Comparativa entre Euler y valor real	6
2.	Diagrama de Clases de los problemas OMP	18
3.	Diagrama de Clases de los métodos OMP	19
4.	Brusselator1D 100.000 ecuaciones	23
5.	ABM4 resolviendo Brusselator2D	25
6.	Speedup y Eficiencia - ABM4 Brusselator2D	25
7.	Jerarquía de memorias en CUDA [Beard, 2020]	28
8.	Grids y bloques CUDA [Casas, 2024]	29
9.	Diagrama de Clases CUDA de los problemas	31
10.	Diagrama de Clases de los structs	32
11.	Diagrama de Clases de los Métodos CUDA	34
12.	Comparativa con Brusselator2D 125K entre tecnologías	36
13.	RK4 a través de tamaños de Brusselator1D en CUDA	37
14.	Dependencias entre las clases Graph	38
15.	Comparación de tiempos de graph con ABM en Brusselator2D	39
16.	Ejemplo de shuffle down [Luitjens, 2014]	40
17.	Tiempos de AB4 en Brusselator1D estándar, Shuffle y Grid .	43

Listado de Tablas

1.	Comparativa entre Euler y valor real	6
2.	Tiempos de Brusselator1D con 100000 ecuaciones	23
3.	Brusselator2D con ABM4	24
4.	Comparativa con Brusselator2D 125K entre tecnologías	35
5.	RK4 a través de tamaños de Brusselator1D en CUDA	36
6.	Comparación de tiempos de graph con ABM en Brusselator2D	39
7.	Tiempos de AB4 en Brusselator1D estándar, Shuffle y Grid .	43